

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

**Trường Đại Học Công Nghệ
Thông Tin**



BÁO CÁO NHÓM 14

Môn học: ĐÁNH GIÁ HIỆU NĂNG MẠNG MÁY TÍNH

Mã môn: NT541.Q21

**Đề Tài: NỀN TẢNG MESSAGE STREAMING
REDPANDA**

GVHD: ThS. Đặng Lê Bảo Chương

DANH SÁCH THÀNH VIÊN			
MSSV	Họ tên	Phân công	Đánh giá
23520640	Nguyễn Văn Huy	Tìm hiểu lý thuyết, cài đặt thực hiện thực nghiệm kịch bản	100%
23521153	Hồ Thị Hữu Phi	Tìm hiểu lý thuyết, nội dung kịch bản, chạy kịch bản	100%

MỤC LỤC

DANH MỤC VIẾT TẮT	6
DANH MỤC HÌNH ẢNH	7
DANH MỤC BẢNG	8
NỘI DUNG BÁO CÁO	9
CHƯƠNG I. Giới thiệu.....	9
1.1. Bối cảnh bài toán.....	9
1.2. Mục tiêu nghiên cứu	9
1.3. Câu hỏi nghiên cứu	9
1.4. Đóng góp của báo cáo	9
CHƯƠNG II. Cơ sở lý thuyết.....	10
2.1. Kiến trúc hướng sự kiện và event streaming	10
2.2. Kiến trúc Redpanda.....	10
2.3. Luồng hoạt động của hệ thống streaming trong Redpanda	11
2.4. Các cơ chế ảnh hưởng đến hiệu năng	12
2.5. Offset, consumer group và backlog.....	13
2.6. Lý thuyết hàng đợi dùng để phân tích hệ thống	13
CHƯƠNG III. Chỉ số đánh giá và phương pháp đo	14
3.1. Throughput.....	14
3.2. Latency	14
3.3. Consumer lag / backlog	14
3.4. Phương pháp thu thập số liệu	15
CHƯƠNG IV. Môi trường và phương pháp thực nghiệm.....	15
4.1. Môi trường thực nghiệm	15
4.2. Các biến thực nghiệm	15
4.3. Quy trình thực nghiệm	16
4.4. Các mối đe dọa đến tính hợp lý	16

CHƯƠNG V. Các bước cài đặt.....	16
5.1. Cài đặt các gói cần thiết trên cả 3 máy ảo VM	16
5.2. Tối ưu hoá hệ thống	16
5.3. Cấu hình và bootstrap cluster.....	16
5.4. Khởi động và kiểm tra cluster	18
5.5 Cài đặt Java và Apache Kafka CLI trên máy Windows.....	18
5.6. Phân tích công cụ và script thực nghiệm	19
5.6.1. Công cụ kafka-producer-perf-test.....	19
5.6.2. Phân tích script kịch bản 1.....	20
5.6.3. Phân tích script kịch bản 2 — Burst traffic	21
5.6.4. Phân tích cơ chế mô phỏng suy giảm mạng — tc netem	21
5.6.5. Phân tích script theo dõi consumer lag — script5 và script6	22
CHƯƠNG VI. Kịch bản thực nghiệm 1: Tải tăng dần và điểm bão hòa.....	23
6.1. Mục tiêu	23
6.2. Thiết lập	23
6.3. Kỳ vọng	24
6.4. Kết quả và phân tích.....	24
6.4.1. Kết quả của nhóm 256B	26
6.4.2. Kết quả của nhóm 1KB	26
6.4.3. Kết quả bổ sung quanh vùng knee point của nhóm 1KB.....	27
6.4.4. Failure case và dấu hiệu quá tải	27
6.4.5. Nhận xét tổng hợp cho kịch bản 1	27
CHƯƠNG VII. Kịch bản thực nghiệm 2: Burst traffic và suy giảm mạng.....	28
7.1. Mục tiêu	28
7.2. Thiết lập	28

7.3. Kỳ vọng	29
7.4. Kết quả và phân tích.....	30
7.4.1. Ảnh hưởng của suy giảm mạng đến throughput	31
7.4.2. Ảnh hưởng của suy giảm mạng đến tail latency	32
7.4.3. Ảnh hưởng của suy giảm mạng đến average latency	32
7.4.4. So sánh giữa acks=1 và acks=all.....	32
7.4.5. Nhận xét tổng hợp cho kịch bản 2	33
CHƯƠNG VIII. Thảo luận tổng hợp	33
8.1. Tác động của tải đến throughput và latency	33
8.2. Tác động của mạng đến cơ chế replication	34
8.3. Trade-off giữa hiệu năng và độ tin cậy	34
CHƯƠNG IX. Kết luận	34
9.1. Kết luận chính	34
9.2. Hạn chế	35
9.3. Hướng phát triển.....	35
CHƯƠNG X. So sánh Redpanda với Apache Kafka và RabbitMQ	36
10.1. Tổng quan	36
10.2. So sánh tổng hợp	36
10.3. Lý do lựa chọn Redpanda	36
TÀI LIỆU THAM KHẢO.....	38

DANH MỤC VIẾT TẮT

Bảng 1. Danh mục các từ viết tắt

[illegible]

DANH MỤC HÌNH ẢNH

Hình 1. Mô hình tổng quát của kiến trúc hướng sự kiện với producer, hệ thống streaming và consumer.	10
Hình 2. Kiến trúc Redpanda gồm cluster, broker, topic, partition và replica.	11
Hình 3. Luồng ghi dữ liệu từ producer đến leader và quá trình replication đến các follower.	12
Hình 4. Luồng đọc dữ liệu của consumer theo offset trong partition.	12
Hình 5. So sánh cơ chế xác nhận dữ liệu trong hai cấu hình acks=1 và acks=all.	13
Hình 6. Ảnh xạ mô hình hàng đợi vào hệ thống streaming: producer là nguồn sinh tải, partition/broker là máy phục vụ, còn backlog phản ánh hàng đợi tồn đọng.	14
Hình 7. Topology logic của môi trường thực nghiệm gồm một máy Windows vật lý, mạng ảo VMware VMnet8 và ba broker Redpanda chạy trên máy ảo. Network impairment được áp trên vùng lưu lượng replication giữa các broker.	15
Hình 8. Tập cấu hình redpanda.yaml trên broker 1 sau khi bootstrap cluster.	17
Hình 9. Kết quả kiểm tra thông tin cụm Redpanda bằng lệnh rpk cluster info.	18
Hình 10. Throughput thực tế theo offered load.	25
Hình 11. P99 latency theo offered load.	25
Hình 12. So sánh throughput trung bình giữa các điều kiện thực nghiệm.	30
Hình 13. So sánh tail latency giữa các điều kiện thực nghiệm.	31
Hình 14. So sánh average latency giữa các điều kiện thực nghiệm.	31

DANH MỤC BẢNG

<i>Bảng 1. Danh mục các từ viết tắt.....</i>	<i>6</i>
<i>Bảng 2. Kết quả đại diện của nhóm 256B.....</i>	<i>26</i>
<i>Bảng 3. Trình bày hai lần chạy đại diện của nhóm bản tin 1KB ở vùng gần và vượt ngưỡng bão hòa.</i>	<i>26</i>
<i>Bảng 4. Kết quả bổ sung quanh vùng knee point của nhóm 1KB.</i>	<i>27</i>
<i>Bảng 5. Trình bày giá trị trung bình của bốn điều kiện thực nghiệm trong kịch bản 2.....</i>	<i>30</i>
<i>Bảng 6. So sánh tổng hợp giữa Apache Kafka, RabbitMQ và Redpanda.....</i>	<i>36</i>

NỘI DUNG BÁO CÁO

CHƯƠNG I. Giới thiệu

1.1. Bối cảnh bài toán

Trong các hệ thống phân tán hiện nay, dữ liệu thường được sinh ra liên tục và cần được xử lý gần thời gian thực. Các bài toán như thu thập log, giám sát hệ thống, tích hợp vi dịch vụ và xử lý giao dịch đều cần một nền tảng có khả năng truyền, lưu trữ và phân phối dữ liệu với thông lượng cao và độ trễ thấp. Vì vậy, các nền tảng message streaming ngày càng đóng vai trò quan trọng trong kiến trúc hướng sự kiện.

Redpanda là một nền tảng event streaming tương thích Kafka API, được thiết kế để tối ưu hiệu năng và đơn giản hóa vận hành. Tuy nhiên, hiệu năng của Redpanda không chỉ phụ thuộc vào bản thân phần mềm mà còn chịu ảnh hưởng bởi tải vào, cơ chế replication và điều kiện mạng giữa các broker. Do đó, Redpanda là một đối tượng phù hợp để nghiên cứu trong môn Đánh giá hiệu năng hệ thống mạng máy tính

1.2. Mục tiêu nghiên cứu

Báo cáo này tập trung vào ba mục tiêu chính:

- Đánh giá throughput và latency của Redpanda dưới các mức tải khác nhau.
- Xác định knee point và ngưỡng bão hòa của hệ thống.
- Phân tích ảnh hưởng của suy giảm mạng và so sánh hai cấu hình acks=1 và acks=all.

1.3. Câu hỏi nghiên cứu

Từ các mục tiêu trên, báo cáo đặt ra các câu hỏi sau:

- RQ1: Redpanda đạt knee point ở mức tải nào khi offered load tăng dần?
- RQ2: Suy giảm mạng ảnh hưởng như thế nào đến throughput và tail latency?
- RQ3: Cấu hình acks=all nhạy với suy giảm mạng hơn acks=1 ra sao?

1.4. Đóng góp của báo cáo

Trong phạm vi đồ án, nhóm triển khai một cụm Redpanda gồm ba broker trên môi trường ảo hóa và thực hiện hai kịch bản chính. Kịch bản thứ nhất khảo sát tải tăng dần để xác định điểm bão hòa. Kịch bản thứ hai khảo sát burst traffic kết hợp với network impairment để phân tích tác động của mạng và cơ chế xác nhận dữ liệu.

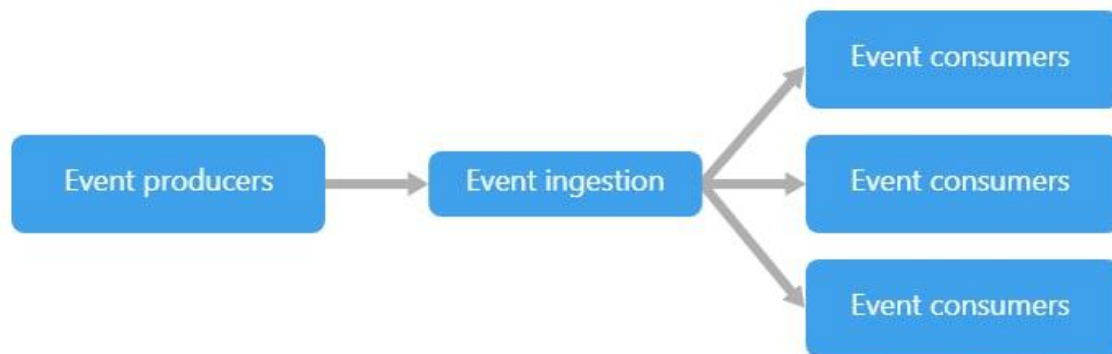
Báo cáo chủ yếu sử dụng các chỉ số như throughput, average latency, p99 và p99.9 để phân tích hiệu năng hệ thống.

CHƯƠNG II. Cơ sở lý thuyết

2.1. Kiến trúc hướng sự kiện và event streaming

Kiến trúc hướng sự kiện là mô hình trong đó các thành phần của hệ thống giao tiếp với nhau thông qua các sự kiện thay vì gọi trực tiếp theo kiểu request/response. Mỗi sự kiện biểu diễn một thay đổi trạng thái hoặc một hành động đã xảy ra, ví dụ như một giao dịch được tạo, một cảm biến gửi dữ liệu, hoặc một đơn hàng được xác nhận. Trong mô hình này, producer phát sinh sự kiện, còn consumer tiếp nhận và xử lý sự kiện một cách bất đồng bộ.

Event streaming là cách tổ chức và truyền các sự kiện đó dưới dạng một dòng dữ liệu liên tục theo thời gian. Nền tảng streaming đóng vai trò trung gian để tiếp nhận, lưu trữ và phân phối dữ liệu đến nhiều consumer khác nhau. Nhờ vậy, producer và consumer được tách rời, giúp hệ thống dễ mở rộng, linh hoạt hơn và phù hợp với các bài toán dữ liệu thời gian thực.



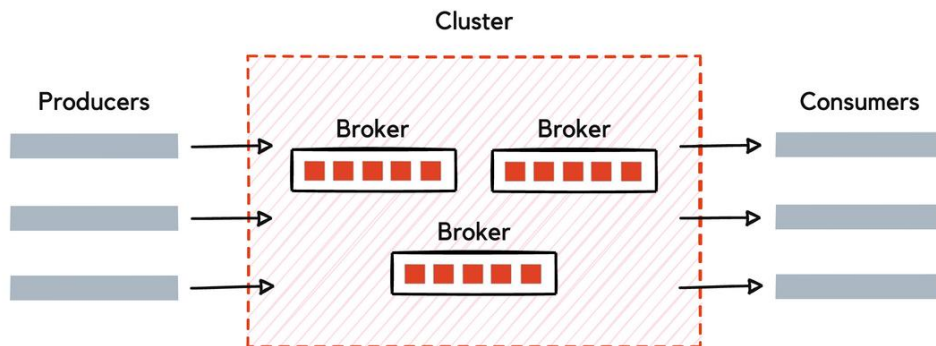
Hình 1. Mô hình tổng quát của kiến trúc hướng sự kiện với producer, hệ thống streaming và consumer.

Trong đồ án này, Redpanda được xem như nền tảng streaming trung gian, nơi dữ liệu được đưa vào từ producer, lưu trữ theo log và sau đó được consumer đọc lại để xử lý. Từ góc nhìn đánh giá hiệu năng, đây là một hệ thống mạng-phân tán điển hình, vì hiệu năng của nó phụ thuộc đồng thời vào tốc độ đưa dữ liệu vào, khả năng xử lý của broker và điều kiện truyền dữ liệu giữa các nút.

2.2. Kiến trúc Redpanda

Redpanda là một nền tảng event streaming tương thích Kafka API, được thiết kế để xử lý dữ liệu thời gian thực với hiệu năng cao. Trong Redpanda, dữ liệu được tổ chức thành các topic. Mỗi topic được chia thành nhiều partition, và mỗi partition là đơn vị lưu trữ và xử lý cơ bản của hệ thống. Việc chia topic thành nhiều partition cho phép hệ thống ghi và đọc dữ liệu song song, từ đó tăng khả năng mở rộng và thông lượng.

Một cụm Redpanda bao gồm nhiều broker. Mỗi broker lưu trữ dữ liệu của một số partition và tham gia vào quá trình replication. Để tăng độ bền dữ liệu, mỗi partition có thể được sao chép thành nhiều bản sao theo replication factor. Trong số các replica của một partition, sẽ có một leader và các follower. Leader là nút tiếp nhận yêu cầu ghi từ producer, còn follower nhận dữ liệu sao chép từ leader để đảm bảo dữ liệu không bị mất khi có lỗi xảy ra.



Hình 2. Kiến trúc Redpanda gồm cluster, broker, topic, partition và replica.

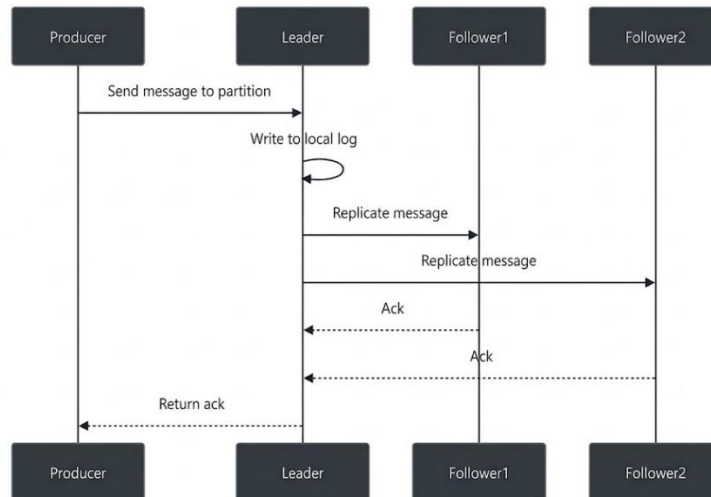
Từ góc nhìn hiệu năng, ba thành phần quan trọng nhất là partition, replica và leader/follower. Partition quyết định mức độ song song hóa của hệ thống. Replication factor quyết định số lượng bản sao dữ liệu và ảnh hưởng trực tiếp đến lưu lượng replication. Cơ chế leader/follower quyết định đường đi của dữ liệu và độ trễ xác nhận trong quá trình ghi.

2.3. Luồng hoạt động của hệ thống streaming trong Redpanda

Luồng hoạt động của Redpanda có thể được chia thành ba giai đoạn chính: ghi dữ liệu từ producer, sao chép dữ liệu giữa các broker, và đọc dữ liệu từ consumer.

Trước hết, producer tạo bản tin và gửi vào một topic. Mỗi bản tin thường gồm key và value. Nếu producer có chỉ định key, hệ thống sẽ dùng key đó để xác định partition tương ứng; nếu không có key, dữ liệu thường được phân phối theo kiểu round-robin giữa các partition. Việc xác định partition là bước quan trọng vì thứ tự dữ liệu chỉ được đảm bảo trong phạm vi một partition.

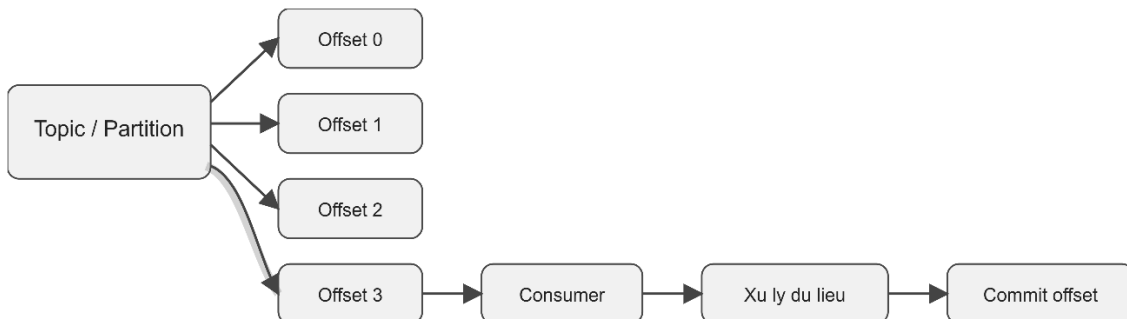
Sau khi chọn partition, bản tin được gửi đến broker đang giữ vai trò leader của partition đó. Leader tiếp nhận dữ liệu, ghi vào log cục bộ, sau đó thực hiện replication đến các follower. Khi đủ điều kiện xác nhận, leader mới phản hồi thành công cho producer.



Hình 3. Luồng ghi dữ liệu từ producer đến leader và quá trình replication đến các follower.

Với $acks=all$, producer chỉ nhận xác nhận sau khi leader thu thập đủ phản hồi từ các replica cần thiết. Vì vậy, độ trễ mạng giữa các broker ảnh hưởng trực tiếp đến độ trễ đầu cuối.

Trong quá trình đọc dữ liệu, consumer subscribe vào topic và đọc bản tin từ các partition được phân công. Mỗi bản tin trong partition có một offset duy nhất. Consumer đọc dữ liệu theo thứ tự offset tăng dần. Sau khi xử lý xong, consumer có thể commit offset để lưu lại vị trí đã đọc, từ đó có thể tiếp tục từ đúng điểm đó nếu ứng dụng bị dừng hoặc khởi động lại.



Hình 4. Luồng đọc dữ liệu của consumer theo offset trong partition.

Consumer đọc dữ liệu theo thứ tự offset tăng dần trong partition. Sau khi xử lý xong, consumer commit offset để lưu lại vị trí đã đọc.

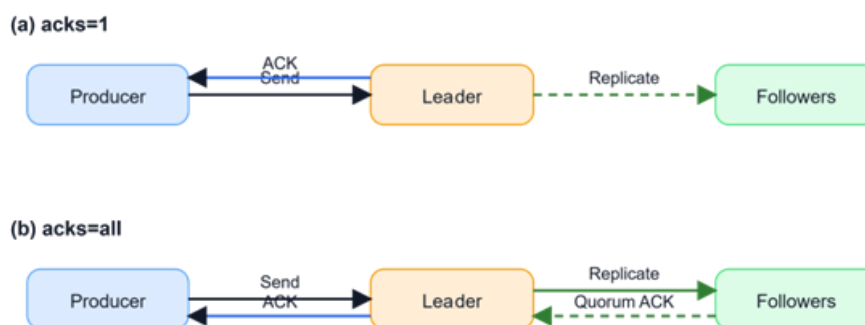
Phân tích theo luồng hoạt động cho thấy hiệu năng của Redpanda không chỉ phụ thuộc vào tốc độ ghi cục bộ tại leader, mà còn phụ thuộc vào toàn bộ đường đi của dữ liệu: từ producer, qua leader, qua follower, rồi đến consumer. Đây là cơ sở để giải thích vì sao chất lượng mạng và cấu hình xác nhận dữ liệu ảnh hưởng mạnh đến throughput và latency của hệ thống.

2.4. Các cơ chế ảnh hưởng đến hiệu năng

Một cơ chế rất quan trọng trong Redpanda là acks. Đây là tham số xác định producer cần chờ loại xác nhận nào trước khi xem một yêu cầu ghi là hoàn tất.

Với $acks=1$, producer chỉ cần chờ leader xác nhận đã nhận dữ liệu. Khi đó, độ trễ xác nhận thường thấp hơn vì producer không phải chờ quá trình sao chép hoàn tất trên các follower. Tuy nhiên, dữ liệu có thể bị mất nếu leader gặp lỗi trước khi replication hoàn tất.

Với $acks=all$, producer chỉ nhận xác nhận sau khi leader thu thập đủ phản hồi từ đa số replica cần thiết. Cấu hình này làm tăng độ bền dữ liệu, nhưng đồng thời cũng làm tăng độ trễ và giảm thông lượng, đặc biệt khi mạng giữa các broker bị suy giảm.



Hình 5. So sánh cơ chế xác nhận dữ liệu trong hai cấu hình $acks=1$ và $acks=all$.

Từ đây có thể thấy rằng trong cấu hình $acks=all$, mạng nội bộ giữa các broker trở thành một phần trực tiếp của đường găng xử lý. Nếu RTT tăng, loss tăng hoặc băng thông bị giới hạn, thời gian replication tăng lên, dẫn đến độ trễ xác nhận cho producer cũng tăng theo. Đây là cơ sở để lý giải vì sao trong các kịch bản thực nghiệm, cấu hình $acks=all$ thường nhạy hơn với suy giảm mạng so với $acks=1$.

2.5. Offset, consumer group và backlog

Trong Redpanda, mỗi bản tin sau khi được ghi vào partition sẽ được gán một offset duy nhất. Offset cho phép consumer biết đang đọc đến vị trí nào trong log. Khi nhiều consumer cùng thuộc một consumer group, các partition sẽ được chia giữa các consumer để xử lý song song.

Khi consumer xử lý dữ liệu chậm hơn tốc độ producer đưa dữ liệu vào hệ thống, khoảng cách giữa offset mới nhất của broker và offset đã commit của consumer sẽ tăng lên. Khoảng cách này thường được gọi là consumer lag hoặc backlog. Đây là chỉ số quan trọng để phản ánh mức độ tồn đọng của hệ thống, đặc biệt trong các kịch bản burst traffic hoặc khi hệ thống bị suy giảm hiệu năng do mạng.

2.6. Lý thuyết hàng đợi dùng để phân tích hệ thống

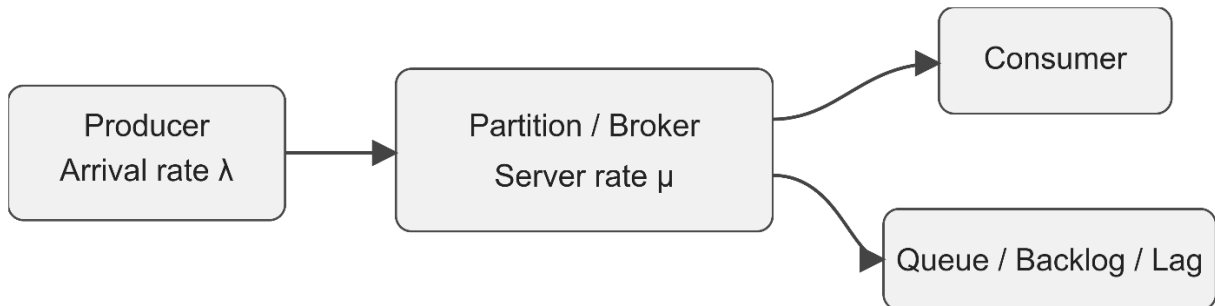
Để phân tích hành vi của Redpanda dưới tải, có thể xem mỗi partition như một hàng đợi phục vụ. Producer tạo ra dòng bản tin đến hệ thống với tốc độ λ , còn broker xử lý dữ liệu với tốc độ phục vụ μ . Khi đó, hệ số sử dụng của hệ thống được biểu diễn bởi $\rho = \lambda / \mu$.

Khi ρ còn thấp, hệ thống hoạt động ổn định, hàng đợi nhỏ và độ trễ ở mức thấp. Khi ρ tiến gần 1, hàng đợi tích tụ nhanh hơn và độ trễ bắt đầu tăng mạnh. Đây là cơ sở để giải thích hiện tượng knee point, tức là điểm mà throughput gần đạt cực đại nhưng tail latency như p95, p99 hoặc p99.9 tăng phi tuyến.

Một công cụ quan trọng khác là định luật Little:

$$L = \lambda W$$

Trong đó, L là số lượng bản tin tồn đọng trong hệ thống, λ là tốc độ đến, và W là thời gian lưu lại trong hệ thống. Trong ngữ cảnh streaming, L có thể được liên hệ với backlog hoặc consumer lag, còn W có thể được liên hệ với latency. Công thức này cho thấy khi tốc độ vào tăng nhưng năng lực xử lý không tăng tương ứng, độ trễ tăng sẽ kéo theo lượng tồn đọng tăng.



Hình 6. Ảnh xạ mô hình hàng đợi vào hệ thống streaming: producer là nguồn sinh tải, partition/broker là máy phục vụ, còn backlog phản ánh hàng đợi tồn đọng.

Từ góc nhìn đánh giá hiệu năng, lý thuyết hàng đợi giúp giải thích vì sao throughput có thể tăng đến một mức rồi gần như không tăng thêm, trong khi latency lại tăng mạnh. Điều này đặc biệt phù hợp với các kịch bản thực nghiệm trong báo cáo, nơi hệ thống được khảo sát dưới tải tăng dần và dưới điều kiện mạng suy giảm.

CHƯƠNG III. Chỉ số đánh giá và phương pháp đo

3.1. Throughput

Throughput là lượng dữ liệu hệ thống xử lý thành công trong một đơn vị thời gian, thường được biểu diễn theo MB/s hoặc records/s. Đây là chỉ số phản ánh khả năng tiếp nhận và xử lý dữ liệu của Redpanda dưới các mức tải khác nhau. Trong báo cáo, throughput được dùng để đánh giá khả năng chịu tải và xác định ngưỡng bão hòa của hệ thống.

3.2. Latency

Latency là thời gian từ khi producer gửi bản tin đến khi nhận được xác nhận thành công từ broker. Báo cáo sử dụng average latency, p50, p95, p99 và p99.9 để phản ánh đầy đủ hành vi của hệ thống. Trong đó, các chỉ số p99 và p99.9 đặc biệt quan trọng vì thể hiện tail latency, tức các trường hợp chậm nhất khi hệ thống tiến gần vùng bão hòa.

3.3. Consumer lag / backlog

Consumer lag là độ chênh lệch giữa offset mới nhất trên broker và offset đã được consumer commit. Chỉ số này phản ánh lượng dữ liệu tồn đọng trong hệ thống và thường tăng lên khi tốc độ dữ liệu vào lớn hơn khả năng xử lý. Trong báo cáo, lag được xem là chỉ số hỗ trợ để diễn giải hiện tượng backlog trong các kịch bản burst traffic.

3.4. Phương pháp thu thập số liệu

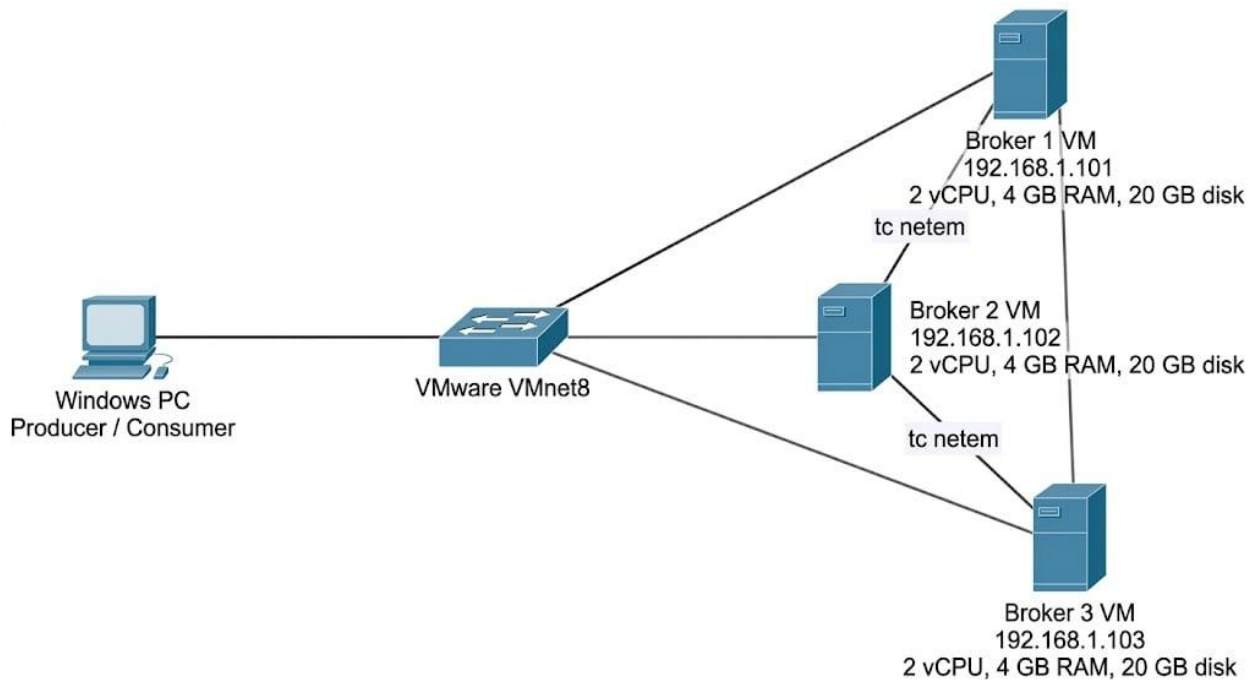
Nhóm sử dụng công cụ benchmark tương thích Kafka API để phát sinh tải và thu thập các chỉ số như throughput, average latency và các percentile độ trễ. Các số liệu được lưu dưới dạng log văn bản để phục vụ cho việc tổng hợp bảng kết quả và phân tích sau thực nghiệm.

CHƯƠNG IV. Môi trường và phương pháp thực nghiệm

4.1. Môi trường thực nghiệm

Hệ thống thực nghiệm được triển khai trên môi trường ảo hóa VMware. Cụm Redpanda gồm 3 broker chạy trên ba máy ảo độc lập. Mỗi máy ảo được cấp phát 2 vCPU, 4 GB RAM và 20 GB dung lượng lưu trữ. Ba broker được cấu hình với các địa chỉ 192.168.1.101, 192.168.1.102 và 192.168.1.103, cùng kết nối trong một mạng ảo nội bộ thông qua VMware VMnet8.

Ngoài cụm broker, nhóm sử dụng một máy Windows vật lý làm máy điều khiển thực nghiệm. Máy này được dùng để phát sinh tải từ phía producer và trong một số kịch bản cũng được dùng để chạy consumer nhằm hỗ trợ quan sát quá trình đọc dữ liệu và thu thập kết quả.



Hình 7. Topology logic của môi trường thực nghiệm gồm một máy Windows vật lý, mạng ảo VMware VMnet8 và ba broker Redpanda chạy trên máy ảo. Network impairment được áp trên vùng lưu lượng replication giữa các broker.

4.2. Các biến thực nghiệm

Các biến độc lập trong báo cáo gồm offered load, kích thước bản tin, cấu hình xác nhận dữ liệu (acks=1, acks=all) và điều kiện mạng giữa các broker. Các biến phụ thuộc là throughput, average latency và các percentile độ trễ như p99 và p99.9.

Trong quá trình đo, các yếu tố như số lượng broker, topology mạng, môi trường ảo hóa và công cụ phát sinh tải được giữ cố định để việc so sánh giữa các kịch bản được nhất quán hơn.

4.3. Quy trình thực nghiệm

Nhóm thực hiện hai kịch bản chính. Kịch bản thứ nhất tăng tải theo từng mức để xác định knee point và ngưỡng bão hòa của hệ thống. Kịch bản thứ hai sử dụng burst traffic kết hợp với suy giảm mạng để phân tích ảnh hưởng của điều kiện mạng và so sánh giữa hai cấu hình acks=1 và acks=all.

Trong các lần chạy có suy giảm mạng, nhóm sử dụng tc netem trên các broker để mô phỏng việc tăng độ trễ, giới hạn băng thông và thêm mất gói ở mức nhẹ. Các số liệu thu được được lưu lại dưới dạng log văn bản để phục vụ cho bước tổng hợp và phân tích sau thực nghiệm.

4.4. Các mối đe dọa đến tính hợp lý

Một hạn chế của thực nghiệm là hệ thống được triển khai trên môi trường ảo hóa VMware, nên kết quả có thể chịu ảnh hưởng bởi overhead của hypervisor và tài nguyên dùng chung trên máy chủ vật lý. Ngoài ra, topology mạng trong môi trường ảo hóa chỉ phản ánh mô hình logic của hệ thống, chưa thể hiện đầy đủ đặc tính của hạ tầng mạng vật lý thực tế.

Bên cạnh đó, báo cáo chủ yếu sử dụng các chỉ số ở tầng ứng dụng như throughput và latency, trong khi một số chỉ số mức hệ thống như CPU, disk I/O hoặc retransmission chưa được thu thập đầy đủ. Vì vậy, các kết luận trong báo cáo được hiểu trong phạm vi của môi trường thực nghiệm hiện tại.

CHƯƠNG V. Các bước cài đặt

5.1. Cài đặt các gói cần thiết trên cả 3 máy ảo VM

Redpanda được cài đặt trực tiếp trên từng máy ảo Ubuntu thông qua repository chính thức. Trên cả ba máy (192.168.1.101, 192.168.1.102, 192.168.1.103), nhóm thực hiện lần lượt các lệnh sau:

```
curl -sLf
'https://dl.redpanda.com/nzc4ZYQK3WRGd9sy/redpanda/cfg/setup
/bash.deb.sh' \ | sudo -E bash

sudo apt install redpanda -y
```

Lệnh trên tự động thêm repository của Redpanda vào hệ thống và cài đặt gói phần mềm kèm theo các dependency cần thiết.

5.2. Tối ưu hoá hệ thống

Trước khi khởi động cluster, Redpanda cung cấp công cụ rpk để tự động điều chỉnh các tham số kernel của hệ điều hành nhằm tối ưu hiệu năng I/O và mạng. Nhóm thực hiện lệnh sau trên cả ba máy:

```
sudo rpk redpanda mode production

sudo rpk redpanda tune all
```

5.3. Cấu hình và bootstrap cluster

- Trên máy broker 0 (192.168.1.101):


```
sudo rpk redpanda config bootstrap --self 192.168.1.101 --
ips 192.168.1.101,192.168.1.102,192.168.1.103

sudo rpk redpanda config set
redpanda.empty_seed_starts_cluster false
```

- Trên máy broker 1 (192.168.1.102):

```
sudo rpk redpanda config bootstrap --self 192.168.1.102 --
ips 192.168.1.101,192.168.1.102,192.168.1.103

sudo rpk redpanda config set
redpanda.empty_seed_starts_cluster false
```

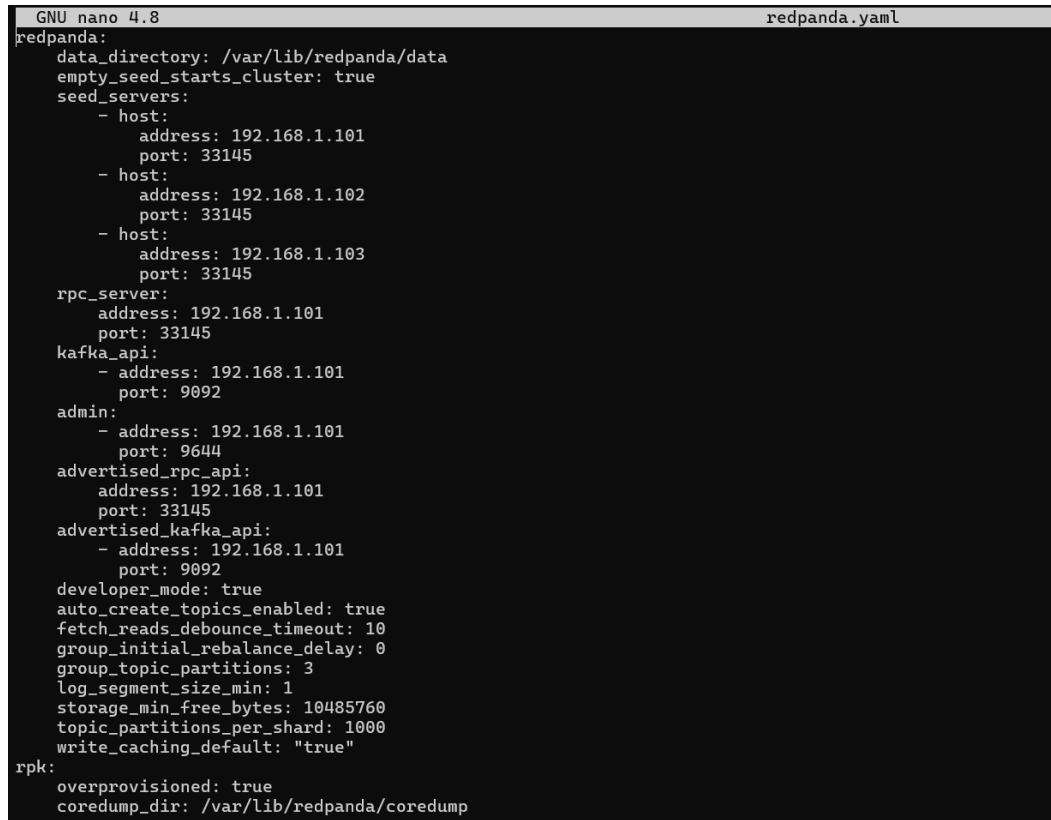
- Trên máy broker 2 (192.168.1.103):

```
sudo rpk redpanda config bootstrap --self 192.168.1.103 --
ips 192.168.1.101,192.168.1.102,192.168.1.103

sudo rpk redpanda config set
redpanda.empty_seed_starts_cluster false
```

Các lệnh trên tự động tạo và ghi file cấu hình `/etc/redpanda/redpanda.yaml` trên từng máy, bao gồm thông tin về seed servers, địa chỉ Kafka API (port 9092), Admin API (port 9644) và RPC nội bộ (port 33145).

- File `redpanda.yaml` trên máy `broker0`:



```
GNU nano 4.8 redpanda.yaml
redpanda:
  data_directory: /var/lib/redpanda/data
  empty_seed_starts_cluster: true
  seed_servers:
    - host:
        address: 192.168.1.101
        port: 33145
    - host:
        address: 192.168.1.102
        port: 33145
    - host:
        address: 192.168.1.103
        port: 33145
  rpc_server:
    address: 192.168.1.101
    port: 33145
  kafka_api:
    - address: 192.168.1.101
      port: 9092
  admin:
    - address: 192.168.1.101
      port: 9644
  advertised_rpc_api:
    address: 192.168.1.101
    port: 33145
  advertised_kafka_api:
    - address: 192.168.1.101
      port: 9092
  developer_mode: true
  auto_create_topics_enabled: true
  fetch_reads_debounce_timeout: 10
  group_initial_rebalance_delay: 0
  group_topic_partitions: 3
  log_segment_size_min: 1
  storage_min_free_bytes: 10485760
  topic_partitions_per_shard: 1000
  write_caching_default: "true"
rpk:
  overprovisioned: true
  coredump_dir: /var/lib/redpanda/coredump
```

Hình 8. Tập cấu hình `redpanda.yaml` trên `broker 1` sau khi bootstrap cluster.

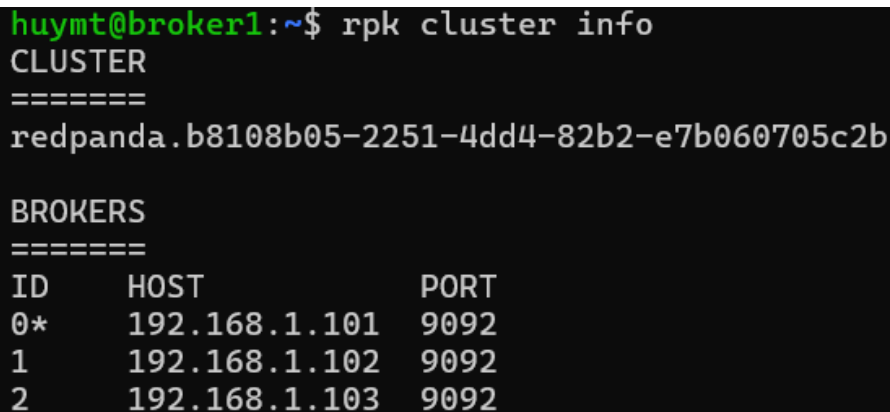
5.4. Khởi động và kiểm tra cluster

Sau khi cấu hình xong cả ba máy, dịch vụ Redpanda được khởi động đồng thời:

```
sudo systemctl start redpanda
sudo systemctl enable redpanda
```

Để xác nhận cluster đã hoạt động đúng, nhóm chạy lệnh kiểm tra trên bất kỳ máy nào:

```
Rpk cluster info
```



```
huytm@broker1:~$ rpk cluster info
CLUSTER
=====
redpanda.b8108b05-2251-4dd4-82b2-e7b060705c2b

BROKERS
=====
ID      HOST          PORT
0*      192.168.1.101 9092
1       192.168.1.102 9092
2       192.168.1.103 9092
```

Hình 9. Kết quả kiểm tra thông tin cụm Redpanda bằng lệnh `rpk cluster info`.

5.5 Cài đặt Java và Apache Kafka CLI trên máy Windows

Ngoài ba máy ảo chạy broker Redpanda, nhóm sử dụng một máy Windows vật lý làm máy điều khiển thực nghiệm. Máy này được dùng để phát sinh tải từ phía producer và trong một số kịch bản cũng được dùng để chạy consumer. Để phục vụ cho quá trình thực nghiệm, nhóm cài đặt Java và bộ công cụ dòng lệnh của Apache Kafka trên hệ điều hành Windows.

Trước hết, nhóm cài đặt Microsoft OpenJDK 11 bằng công cụ winget, sau đó cấu hình biến môi trường JAVA_HOME trỏ đến thư mục cài đặt Java và bổ sung thư mục bin của Java vào biến Path. Tiếp theo, nhóm tải gói Apache Kafka 2.13-3.7.0 từ trang chính thức của Apache Kafka, giải nén vào thư mục D:\Kafka\kafka_2.13-3.7.0, đồng thời bổ sung thư mục bin\windows của Kafka vào biến Path để có thể sử dụng trực tiếp các công cụ như kafka-topics.bat, kafka-console-consumer.bat, kafka-consumer-groups.bat và kafka-producer-perf-test.bat.

Các bước cài đặt chính được thực hiện như sau:

```
winget install --id Microsoft.OpenJDK.11 -e

[System.Environment]::SetEnvironmentVariable("JAVA_HOME",
"C:\Program Files\Microsoft\jdk-11.0.30.7-hotspot",
"Machine")

[System.Environment]::SetEnvironmentVariable("Path",
$env:Path + ";C:\Program Files\Microsoft\jdk-11.0.30.7-
hotspot\bin", "Machine")
```

```
[System.Environment]::SetEnvironmentVariable("Path",  
$env:Path + ";D:\Kafka\kafka_2.13-3.7.0\bin\windows",  
"Machine")
```

Sau khi hoàn tất cấu hình, nhóm kiểm tra lại môi trường bằng các lệnh như `java -version` và `kafka-topics.bat --version` để bảo đảm Java và Kafka CLI đã sẵn sàng cho quá trình đo kiểm.

5.6. Phân tích công cụ và script thực nghiệm

Chương này trình bày phân tích các công cụ, tham số API và script được nhóm sử dụng trong quá trình thực nghiệm. Mục tiêu là làm rõ cơ chế hoạt động của từng thành phần kỹ thuật, giúp người đọc hiểu rõ lý do lựa chọn các tham số cụ thể và cách thiết kế thực nghiệm phản ánh mục tiêu nghiên cứu.

5.6.1. Công cụ kafka-producer-perf-test

Nhóm sử dụng công cụ `kafka-producer-perf-test` thuộc bộ Apache Kafka CLI để phát sinh tải từ phía producer. Đây là công cụ benchmark chính thức của Kafka, tương thích với Redpanda qua Kafka API, cho phép kiểm soát chính xác tốc độ gửi, kích thước bản tin và các tham số producer. Cú pháp lệnh sử dụng trong thực nghiệm như sau:

```
kafka-producer-perf-test.bat  
--topic <ten_topic>  
--num-records <so_ban_tin>  
--record-size <kich_thuoc_byte>  
--throughput <toc_do_msg/s>  
--producer-props  
    bootstrap.servers=<dia_chi_broker>  
    acks=<1|all>  
    linger.ms=<ms>  
    delivery.timeout.ms=<ms>  
    request.timeout.ms=<ms>
```

Phân tích từng tham số quan trọng như sau.

- **--num-records:** Tổng số bản tin producer gửi trong một lần chạy. Trong kịch bản 1, nhóm đặt 200.000 records để thời gian đo đủ dài (tối thiểu 30–60 giây ở mức tải thấp), giúp hệ thống đạt trạng thái ổn định trước khi kết thúc. Trong kịch bản 2, nhóm dùng 20.000 records mỗi burst để phù hợp với chu kỳ 10 giây ON.
- **--record-size:** Kích thước mỗi bản tin tính bằng byte. Kịch bản 1 sử dụng 1.024 bytes (1 KB) và 256 bytes (256 B) để so sánh ảnh hưởng của message size đến knee point. Kịch bản 2 dùng 4.096 bytes (4 KB) để tạo lưu lượng đủ lớn, làm rõ tác động của network impairment.
- **--throughput:** Giới hạn tốc độ gửi tính bằng records/giây. Khi đặt giá trị dương, công cụ điều tiết tốc độ bằng cách chèn độ trễ giữa các batch. Ví dụ: `--throughput 10000` với `record-size 1 KB` tương đương offered load khoảng 9,77 MB/s (10.000×1.024 bytes). Khi đặt -1, công cụ gửi không giới hạn tốc độ, được dùng trong kịch bản 2 để mô phỏng burst traffic tối đa.

- **acks:** Tham số quyết định điều kiện để một yêu cầu ghi được coi là thành công. Với `acks=1`, producer nhận ACK ngay khi leader ghi vào log cục bộ, không chờ replication, nên độ trễ thấp hơn nhưng có nguy cơ mất dữ liệu nếu leader crash trước khi replica đồng bộ. Với `acks=all`, producer chỉ nhận ACK sau khi leader thu đủ xác nhận từ tất cả In-Sync Replicas (ISR), đảm bảo độ bền dữ liệu cao hơn nhưng độ trễ phụ thuộc trực tiếp vào tốc độ replication và chất lượng mạng giữa các broker.
- **linger.ms=5:** Producer đợi tối đa 5 ms trước khi gửi một batch, ngay cả khi batch chưa đầy. Điều này giúp gom nhiều record vào cùng một request, tăng hiệu quả ghi và giảm số lượng request gửi đến broker. Giá trị 5 ms là mức vừa đủ để tạo batching hiệu quả mà không làm tăng đáng kể độ trễ tối thiểu.
- **delivery.timeout.ms:** Thời gian tối đa kể từ khi gửi để một record được xác nhận thành công, bao gồm cả thời gian retry. Nhóm đặt 300.000 ms (5 phút) trong các kịch bản có netem để producer không từ bỏ quá sớm khi mạng chậm. Trong kịch bản 1, giá trị 60.000–120.000 ms là đủ vì mạng ổn định.
- **request.timeout.ms:** Thời gian tối đa cho mỗi request riêng lẻ gửi đến broker. Cần nhỏ hơn `delivery.timeout.ms` để cơ chế retry hoạt động đúng. Nhóm đặt 120.000 ms khi có netem để đủ thời gian chờ broker phản hồi trong điều kiện mạng có thêm 50 ms delay.

5.6.2. Phân tích script kịch bản 1

run_kb1.bat — Tải tăng dần từng mức

Script sử dụng vòng lặp for để lần lượt chạy qua 7 mức tải: 2, 4, 6, 8, 10, 12 và 15 MB/s. Đoạn code trung tâm của script như sau:

```
for %%R in (2 4 6 8 10 12 15) do (
    kafka-producer-perf-test.bat --topic kb1-topic
        --num-records 200000 --record-size 1024
        --throughput %%R000
        --producer-props bootstrap.servers=... acks=1
            linger.ms=5 delivery.timeout.ms=60000
            request.timeout.ms=30000
    > %RESULT_DIR%\1KB_%%RMBps.txt 2>&1
    timeout /t 30 /nobreak > nul
)
```

Biến vòng lặp `%%R` nhận giá trị MB/s, và `%%R000` mở rộng thành records/giây (ví dụ: `%%R = 10` ứng với `--throughput 10000`). Kết quả stdout và stderr được chuyển hướng bằng `2>&1` vào file .txt riêng cho mỗi mức tải. Lệnh `timeout /t 30` tạo khoảng nghỉ 30 giây giữa các mức để cluster Redpanda hồi phục về trạng thái ổn định, tránh hiệu ứng tích lũy ảnh hưởng sang lần đo tiếp theo.

script3_1kb_kneepoint.bat — Đo chi tiết vùng knee point

Script lặp lại mỗi mức tải 10, 11 và 12 MB/s ba lần độc lập, đặt tên file theo biến `run%%R` để phân biệt từng lần chạy. Đoạn lặp cho mức 10 MB/s như sau:

```

for %%R in (1 2 3) do (
    call kafka-producer-perf-test.bat ...
        --throughput 10000
        > "%RESULT_DIR%\1KB_10MBps_run%%R.txt" 2>&1
    echo ERRORLEVEL=!errorlevel!
    timeout /t 30 /nobreak > nul
)

```

Việc lặp lại ba lần mỗi mức giúp phát hiện dao động ngẫu nhiên và phân biệt run hợp lệ với failure case. Khai báo setlocal EnableDelayedExpansion ở đầu script là cần thiết để sử dụng cú pháp !errorlevel! bên trong vòng lặp for — trên Windows, %errorlevel% không cập nhật đúng giá trị khi đứng trong thân vòng lặp.

5.6.3. Phân tích script kịch bản 2 — Burst traffic

Bốn script (7, 8, 9, 10) có cấu trúc đồng nhất, chỉ khác nhau ở biến ACKS và điều kiện mạng. Mỗi script khai báo các tham số cấu hình tập trung ở đầu file để dễ thay đổi:

```

set ACKS=1           :: hoac all
set RECORD_SIZE=4096 :: 4 KB mỗi bản tin
set THROUGHPUT=2000  :: 2000 msg/s
set NUM_RECORDS=20000 :: 20000 / 2000 = 10 giây ON
set BURSTS=3

```

Thời gian mỗi pha ON được tính theo công thức: $\text{NUM_RECORDS} \div \text{THROUGHPUT} = 20.000 \div 2.000 = 10$ giây. Sau mỗi burst, lệnh timeout /t 20 tạo khoảng nghỉ 20 giây (pha OFF) để consumer tiêu thụ phần dữ liệu tồn đọng và hệ thống phục hồi trước chu kỳ tiếp theo.

Trước khi gửi dữ liệu, mỗi script tạo topic với cấu hình cố định:

```

kafka-topics.bat --bootstrap-server %BROKERS%
    --create --if-not-exists --topic %TOPIC%
    --partitions 9 --replication-factor 3

```

Tham số --partitions 9 phân tán đều 9 partition trên 3 broker (mỗi broker đảm nhận 3 partition leader), giúp cân bằng tải ghi. Tham số --replication-factor 3 đảm bảo mỗi partition có đủ replica trên cả 3 node, khiến acks=all phải chờ xác nhận từ toàn bộ ISR — đây là điều kiện cần để thực nghiệm phản ánh đúng sự khác biệt giữa acks=1 và acks=all khi mạng bị suy giảm.

5.6.4. Phân tích cơ chế mô phỏng suy giảm mạng — tc netem

Nhóm sử dụng hai lệnh tc kết hợp trên mỗi VM broker để mô phỏng điều kiện mạng xấu trên interface ens33:

```

sudo tc qdisc add dev ens33 root handle 1: \
    tbf rate 50mbit burst 32kbit latency 400ms

sudo tc qdisc add dev ens33 parent 1:1 handle 10: \
    netem delay 50ms loss 0.1%

```

- **tbf (Token Bucket Filter):** Giới hạn băng thông xuống 50 Mbit/s. tbf hoạt động theo nguyên lý token bucket: mỗi giây hệ thống phát ra lượng token tương đương 50 Mbit; mỗi packet tiêu thụ token tương ứng kích thước của nó; khi hết token, packet phải chờ trong

hàng đợi. Tham số burst 32kbit cho phép gửi tức thì tối đa 32 Kbit mà không cần chờ token, nhằm xử lý các đợt nhỏ không bị nghẽn. Tham số latency 400ms là thời gian trễ tối đa hàng đợi có thể giữ packet trước khi drop.

- **netem (Network Emulator):** Thêm 50 ms trễ cố định và 0,1% mất gói vào mỗi packet ra. netem được gắn làm child qdisc của tbf (tham số parent 1:1), nghĩa là packet đi qua giới hạn băng thông trước rồi mới qua netem. Kết quả là mạng nội bộ giữa các broker đồng thời bị giới hạn băng thông, tăng RTT và có mất gói nhẹ — đủ tạo tác động rõ rệt trong khi cluster vẫn duy trì hoạt động ổn định để đo kiểm.

Để xóa toàn bộ impairment sau thực nghiệm, nhóm chạy lệnh sau trên mỗi VM:

```
sudo tc qdisc del dev ens33 root
```

5.6.5. Phân tích script theo dõi consumer lag — script5 và script6

script5_kb2_start_consumer.bat — Chạy consumer song song

Script khởi động consumer trong một cửa sổ lệnh riêng, chạy song song với producer:

```
kafka-consumer-perf-test.bat
--topic %TOPIC%
--bootstrap-server %BROKERS%
--messages 9999999
--group %GROUP_ID%
--timeout 60000
--reporting-interval 5000
--show-detailed-stats
--from-latest
> consumer_log.txt 2>&1
```

Tham số --from-latest khiến consumer chỉ đọc các bản tin được ghi sau khi nó khởi động, tránh tiêu thụ dữ liệu cũ từ các lần chạy trước. Tham số --reporting-interval 5000 in ra thống kê tiêu thụ mỗi 5 giây, cho phép quan sát throughput của consumer theo thời gian thực. Tham số --messages 9999999 đặt giới hạn đủ lớn để consumer chạy liên tục suốt kịch bản mà không tự dừng.

script6_kb2_monitor_lag.bat — Poll consumer lag định kỳ

Script lấy mẫu trạng thái consumer group theo chu kỳ đều đặn:

```
set SAMPLE_INTERVAL=2      :: 2 giây mỗi mẫu
set NUM_SAMPLES=120        :: tổng 240 giây

for /L %%I in (1,1,%NUM_SAMPLES%) do (
    echo ===== SAMPLE %%I ===== >> lag_file.txt
    echo DATE !date! TIME !time! >> lag_file.txt
    kafka-consumer-groups.bat
        --bootstrap-server %BROKERS%
        --describe --group %GROUP_ID%
        --topic %TOPIC% --timeout 3000
    >> lag_file.txt 2>&1
    timeout /t %SAMPLE_INTERVAL% /nobreak > nul
)
```

Mỗi mẫu gọi lệnh `kafka-consumer-groups --describe` để truy vấn broker về trạng thái consumer group. Kết quả trả về bao gồm ba cột quan trọng: `LOG-END-OFFSET` là offset bản tin mới nhất trên broker, `CURRENT-OFFSET` là offset mà consumer đã commit, và `LAG` là hiệu số giữa hai giá trị đó. Khi producer ghi nhanh hơn consumer tiêu thụ, `LAG` tăng dần; trong pha OFF của burst, `LAG` giảm dần khi consumer bắt kịp. Chuỗi 120 mẫu $\times$ 2 giây = 240 giây cho phép quan sát đầy đủ ít nhất 4–5 chu kỳ ON/OFF, đủ để nhận thấy sự khác biệt về tốc độ recovery giữa các điều kiện thực nghiệm.

CHƯƠNG VI. Kịch bản thực nghiệm 1: Tải tăng dần và điểm bão hòa

6.1. Mục tiêu

Kịch bản thực nghiệm thứ nhất được xây dựng nhằm khảo sát khả năng chịu tải của cụm Redpanda khi offered load tăng dần theo từng mức. Mục tiêu chính của kịch bản này là xác định knee point, tức vùng mà throughput của hệ thống bắt đầu bão hòa trong khi latency tăng mạnh theo hướng phi tuyến. Đây là dấu hiệu quan trọng để nhận biết giới hạn vận hành an toàn của hệ thống.

Ngoài việc xác định ngưỡng bão hòa, kịch bản này còn nhằm quan sát ảnh hưởng của message size đến hiệu năng. Trong phạm vi đề án, nhóm sử dụng hai nhóm kích thước bản tin là 256B và 1KB. Việc thay đổi kích thước bản tin giúp đánh giá xem khi tải tăng lên, hệ thống sẽ phản ứng khác nhau ra sao về throughput và latency. Từ đó, nhóm có thể phân tích liệu sự thay đổi của offered load chỉ làm giảm hiệu năng theo hướng tuyến tính, hay dẫn đến hiện tượng queue buildup và tail latency tăng đột biến.

Về mặt đánh giá hiệu năng hệ thống mạng, kịch bản này có ý nghĩa quan trọng vì nó cho thấy mối quan hệ giữa tốc độ dữ liệu vào hệ thống, năng lực xử lý thực tế của broker và các chỉ số chất lượng dịch vụ ở tầng ứng dụng. Nói cách khác, đây là kịch bản dùng để trả lời câu hỏi: Redpanda bắt đầu mất ổn định ở mức tải nào, và sự mất ổn định đó được phản ánh như thế nào qua throughput và latency.

6.2. Thiết lập

Kịch bản 1 được thực hiện trong điều kiện mạng cơ sở, tức không áp dụng network impairment. Cụm Redpanda gồm ba broker chạy trên môi trường VMware như đã mô tả ở Chương IV. Trong kịch bản này, nhóm sử dụng cấu hình `acks=1` để tập trung quan sát giới hạn xử lý của hệ thống trong điều kiện bình thường, trước khi đưa thêm tác động của mạng ở kịch bản sau.

Các lần chạy được chia thành hai nhóm theo kích thước bản tin:

- Nhóm 256B, với các mức tải từ 2 MB/s đến 12 MB/s.
- Nhóm 1KB, với các mức tải từ 2 MB/s đến 12 MB/s, kèm các lần chạy bổ sung quanh vùng 10–11 MB/s để làm rõ vị trí xuất hiện knee point.

Dữ liệu được thu thập bằng công cụ benchmark tương thích Kafka API. Các chỉ số được ghi nhận gồm throughput, average latency, p50, p95, p99 và p99.9. Trong quá trình phân tích, nhóm phân biệt rõ giữa:

- run hợp lệ: có số liệu tổng kết đầy đủ và phản ánh trạng thái vận hành bình thường hoặc gần bão hòa.
- run lỗi / failure case: xuất hiện TimeoutException, BufferExhaustedException hoặc lỗi metadata nghiêm trọng, được dùng làm bằng chứng cho vùng quá tải nhưng không dùng để tính giá trị đại diện cho đường cong chính.

6.3. Kỳ vọng

Về mặt lý thuyết, khi offered load còn thấp hơn nhiều so với năng lực phục vụ của hệ thống, throughput sẽ tăng gần tuyến tính theo tải vào, còn latency chỉ tăng nhẹ. Trong vùng này, hệ thống vẫn còn đủ tài nguyên để xử lý dữ liệu và hàng đợi nội bộ chưa tích tụ đáng kể.

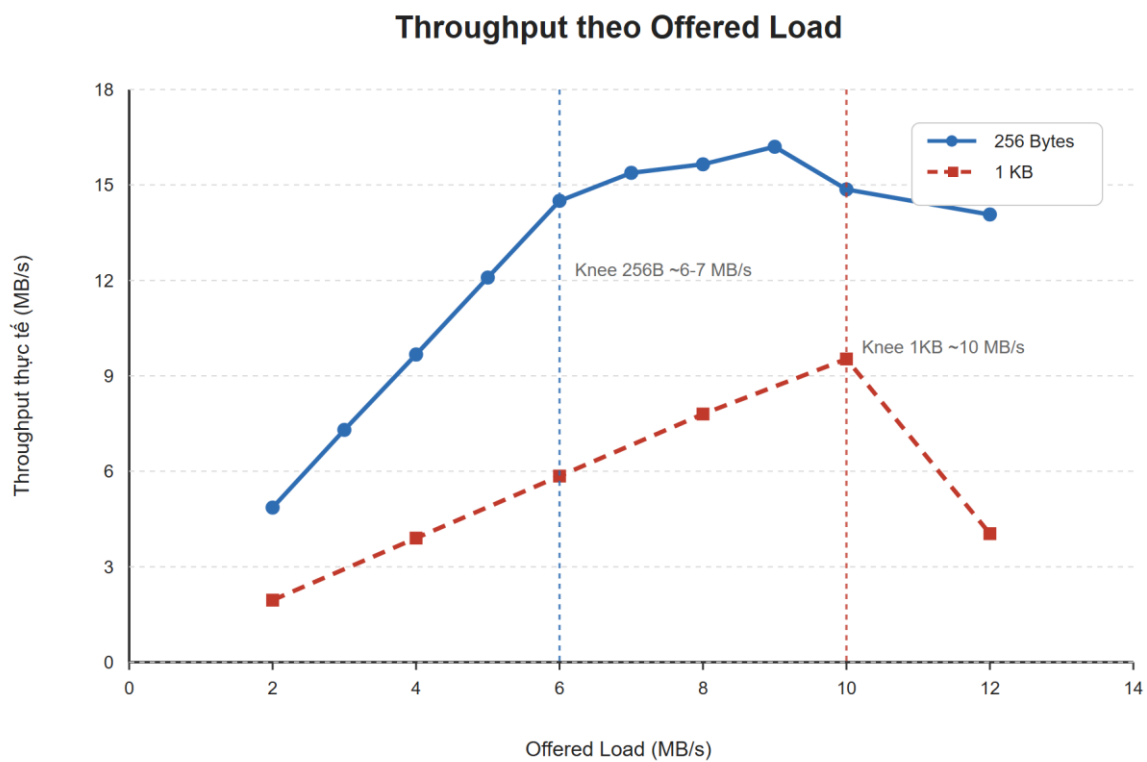
Khi tải tiếp tục tăng và tiến gần đến năng lực phục vụ cực đại, throughput bắt đầu tăng chậm lại hoặc đi ngang. Đây là tín hiệu cho thấy hệ thống đang tiến gần knee point. Tại vùng này, chỉ cần một mức tăng nhỏ về tải cũng có thể làm hàng đợi tích tụ nhanh hơn, dẫn đến average latency tăng mạnh và đặc biệt là p95, p99, p99.9 tăng rõ rệt.

Theo lý thuyết hàng đợi, khi tốc độ đến λ tiến gần tốc độ phục vụ μ , hệ số sử dụng $\rho = \lambda/\mu$ tiến gần 1. Khi đó, thời gian chờ trong hệ thống tăng rất nhanh thay vì tăng tuyến tính. Vì vậy, kỳ vọng chính của kịch bản này là:

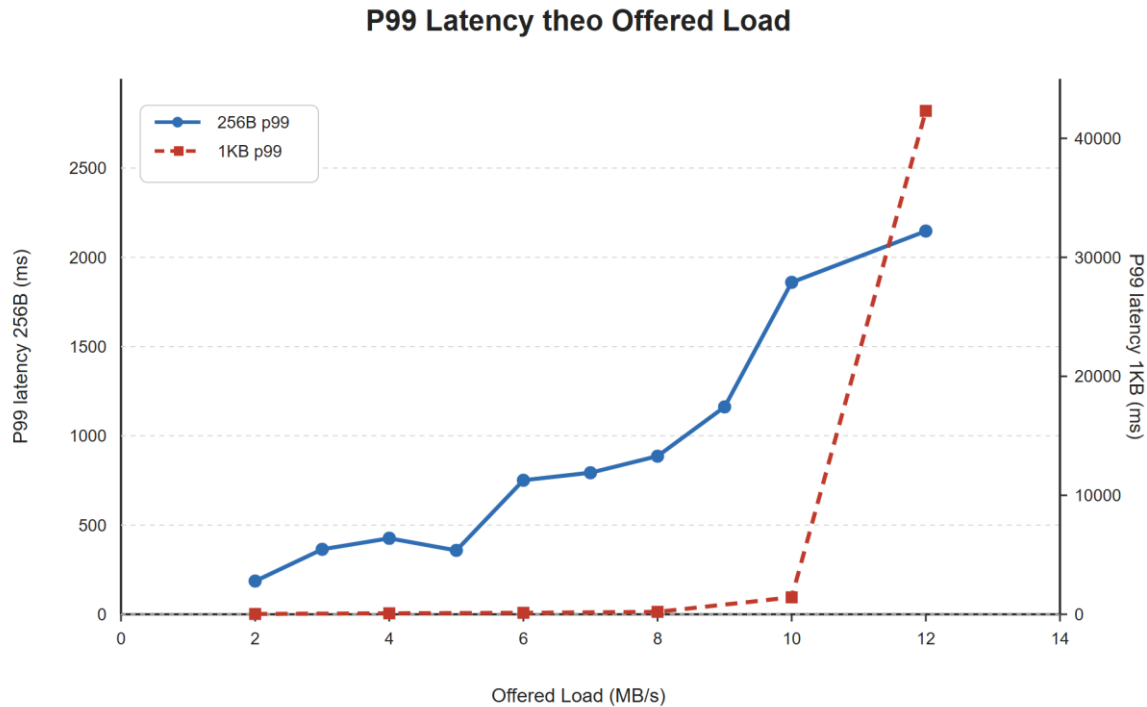
- Throughput tăng dần rồi bão hòa.
- Latency, đặc biệt là tail latency, tăng mạnh ở vùng gần bão hòa.
- Một số lần chạy ở mức tải quá cao có thể xuất hiện timeout hoặc lỗi bất ổn, phản ánh trạng thái overload của hệ thống.

6.4. Kết quả và phân tích

Hình 5.1 và Hình 5.2 trình bày xu hướng tổng quát của throughput và p99 latency khi offered load tăng dần đối với hai nhóm kích thước bản tin 256B và 1KB. Có thể thấy throughput không tăng tuyến tính vô hạn theo tải vào, trong khi p99 latency tăng mạnh khi hệ thống tiến gần vùng bão hòa. Từ hai đồ thị này, nhóm tiếp tục phân tích chi tiết từng nhóm bản tin ở các mục sau.



Hình 10. Throughput thực tế theo offered load.



Hình 11. P99 latency theo offered load.

6.4.1. Kết quả của nhóm 256B

Từ xu hướng tổng quát thể hiện trong Hình 5.1 và Hình 5.2, nhóm tiếp tục xem xét các điểm tiêu biểu của nhóm bản tin 256B nhằm làm rõ dấu hiệu tiến gần vùng bão hòa.

Offered load	Throughput (MB/s)	Avg latency (ms)	p50 (ms)	p95 (ms)	p99 (ms)	p99.9(ms)
6 MB/s	14.50	126.29	38	420	751	1025
7 MB/s	15.38	559.59	576	722	793	842

Bảng 2. Kết quả đại diện của nhóm 256B

Ngoài hai điểm tiêu biểu trong Bảng 5.1, đường cong 256B trong Hình 5.1 cho thấy throughput tăng dần từ 4.86 MB/s tại mức 2 MB/s lên khoảng 16.20 MB/s tại mức 9 MB/s, sau đó bắt đầu đi ngang và giảm nhẹ ở các mức tải cao hơn. Đây là dấu hiệu cho thấy hệ thống không còn tăng throughput tương ứng với tải vào như ở vùng tải thấp.

Kết quả tại 6 MB/s và 7 MB/s cho thấy khi offered load tăng từ 6 MB/s lên 7 MB/s, throughput chỉ tăng nhẹ từ 14.50 MB/s lên 15.38 MB/s, tương đương khoảng 6.1%. Tuy nhiên, average latency tăng từ 126.29 ms lên 559.59 ms, tức tăng khoảng 4.43 lần. Đặc biệt, p50 tăng từ 38 ms lên 576 ms, cho thấy toàn bộ phân bố độ trễ đã dịch chuyển lên mức cao hơn chứ không chỉ phần đuôi.

Từ Hình 5.2, p99 của nhóm 256B tăng từ 186 ms ở mức 2 MB/s lên 2147 ms ở mức 12 MB/s. Mặc dù throughput vẫn được duy trì ở mức khá cao trong vùng tải lớn, độ trễ đuôi đã tăng đáng kể. Điều này cho thấy với nhóm 256B, hệ thống vẫn còn duy trì được thông lượng tương đối tốt, nhưng đã bắt đầu phải đánh đổi bằng việc tăng độ trễ khi tiến gần vùng bão hòa.

Như vậy, đối với 256B, knee point không xuất hiện dưới dạng throughput sụt giảm đột ngột, mà thể hiện rõ hơn qua việc latency tăng nhanh trong khi throughput gần như không tăng thêm đáng kể. Đây là dấu hiệu sớm của saturation.

6.4.2. Kết quả của nhóm 1KB

So với nhóm 256B, nhóm 1KB thể hiện rõ hơn hiện tượng knee point và sự bùng nổ của tail latency, như đã thể hiện trong Hình 5.1 và Hình 5.2.

Offered load	Throughput (MB/s)	Avg latency (ms)	p50 (ms)	p95 (ms)	p99 (ms)	p99.9(ms)
10 MB/s	9.53	220.00	32	1010	1426	1524
12 MB/s	4.04	1801.38	177	1397	42306	43639

Bảng 3. Trình bày hai lần chạy đại diện của nhóm bản tin 1KB ở vùng gần và vượt ngưỡng bão hòa.

Đường cong 1KB trong Hình 5.1 cho thấy throughput tăng khá đều từ 1.95 MB/s tại mức 2 MB/s lên 9.53 MB/s tại mức 10 MB/s, sau đó giảm mạnh xuống 4.04 MB/s tại mức 12 MB/s. Đây là biểu hiện rất rõ của việc hệ thống đã vượt qua vùng vận hành ổn định.

Tại 10 MB/s, hệ thống vẫn đạt được throughput gần bằng offered load, cho thấy Redpanda còn duy trì được khả năng xử lý tốt. Tuy nhiên, khi tăng lên 12 MB/s, throughput giảm khoảng 57.6%, trong khi average latency tăng từ 220.00 ms lên 1801.38 ms, tương đương khoảng 8.19 lần. Đặc biệt, p99 tăng từ 1426 ms lên 42306 ms, tức gần 29.7 lần.

Kết quả này là bằng chứng rất rõ cho hiện tượng knee point. Khi offered load vượt qua năng lực xử lý hiệu dụng của hệ thống, broker không còn duy trì được throughput như trước, trong khi hàng đợi nội bộ tích tụ nhanh làm tail latency tăng đột biến. Từ góc nhìn vận hành, đây là vùng không còn an toàn vì hệ thống vẫn tiếp nhận tải nhưng thời gian phản hồi đã tăng lên mức rất lớn.

6.4.3. Kết quả bổ sung quanh vùng knee point của nhóm 1KB

Để quan sát rõ hơn vùng chuyển tiếp, nhóm thực hiện thêm các lần chạy quanh 10-11 MB/s. Kết quả hợp lệ tiêu biểu được trình bày ở Bảng 5.3.

Run bổ sung	Throughput (MB/s)	Avg latency (ms)	p50 (ms)	p95 (ms)	p99 (ms)	p99.9(ms)
1KB_10MBps_run1	7.56	2430.41	2486	5364	6032	6410
1KB_11MBps_run1	8.13	2232.79	954	8015	15697	15835

Bảng 4. Kết quả bổ sung quanh vùng knee point của nhóm 1KB.

Mặc dù hai run này có dao động nhất định, chúng vẫn cho thấy một xu hướng chung: ở vùng 10–11 MB/s, hệ thống đã bước vào trạng thái rất nhạy cảm. Throughput không tăng tương ứng với offered load, trong khi p95, p99 và p99.9 tăng mạnh. Cụ thể, p99 tăng từ 6032 ms lên 15697 ms, tương đương khoảng 2.6 lần, dù throughput chỉ thay đổi ở mức nhỏ.

Điều này phù hợp với bản chất của knee point: đây không phải là một giá trị tuyệt đối duy nhất mà là một vùng chuyển tiếp, nơi hệ thống bắt đầu mất ổn định và các chỉ số độ trễ tăng rất mạnh khi tải chỉ tăng thêm một lượng nhỏ. Vì vậy, đối với nhóm 1KB, có thể xem vùng 10–11 MB/s là khoảng giá trị mà hệ thống bắt đầu rời khỏi trạng thái vận hành ổn định.

6.4.4. Failure case và dấu hiệu quá tải

Ngoài các run hợp lệ, nhóm cũng ghi nhận một số lần chạy bị lỗi ở vùng tải cao, tiêu biểu là các run xuất hiện TimeoutException, NOT_LEADER_OR_FOLLOWER và BufferExhaustedException. Những trường hợp này không được dùng để tính toán giá trị đại diện, nhưng có ý nghĩa quan trọng trong phân tích.

Các run lỗi cho thấy khi tải vượt quá năng lực hấp thụ ổn định của hệ thống, producer có thể bị đầy bộ đệm, broker có thể phản hồi chậm hoặc mất đồng bộ vai trò leader trên một số partition. Điều này xác nhận rằng vùng quá tải không chỉ làm giảm thông lượng và tăng latency, mà còn có thể làm hệ thống xuất hiện lỗi vận hành thực sự.

Vì vậy, trong bối cảnh của đồ án, các run lỗi được xem là failure under overload, tức bằng chứng cho thấy hệ thống đã đi ra ngoài vùng vận hành an toàn.

6.4.5. Nhận xét tổng hợp cho kịch bản 1

Từ các kết quả trên có thể rút ra ba nhận xét chính.

Thứ nhất, Redpanda vẫn duy trì được throughput tốt khi tải còn trong vùng ổn định, nhưng throughput không tăng mãi theo offered load. Khi hệ thống tiến gần giới hạn xử lý, throughput bắt đầu đi ngang hoặc giảm. Hình 5.1 cho thấy điều này đặc biệt rõ ở nhóm 1KB, nơi throughput đạt cực đại quanh 10 MB/s rồi giảm mạnh tại 12 MB/s.

Thứ hai, sự thay đổi của latency là chỉ báo nhạy hơn throughput trong việc nhận biết knee point. Hình 5.2 cho thấy p99 latency của nhóm 1KB tăng đột biến tại vùng tải cao, trong khi throughput đã bắt đầu suy giảm. Điều này cho thấy nếu chỉ quan sát throughput, người vận hành có thể bỏ qua trạng thái mất ổn định đang hình thành trong hệ thống.

Thứ ba, kích thước bản tin ảnh hưởng rõ rệt đến hành vi bão hòa. Với nhóm 256B, hệ thống vẫn giữ được throughput gần mức cực đại trong một khoảng tải rộng hơn, nhưng latency tăng rõ ở vùng tải cao. Với nhóm 1KB, hiện tượng quá tải rõ ràng hơn, thể hiện qua sự sụt giảm throughput đi kèm với tail latency tăng đột biến và sự xuất hiện của các run lỗi.

Từ góc nhìn đánh giá hiệu năng, kịch bản 1 cho thấy knee point của hệ thống nằm xấp xỉ trong vùng 10–11 MB/s đối với nhóm 1KB, trong khi với nhóm 256B, dấu hiệu bão hòa bắt đầu xuất hiện rõ trong vùng 6–7 MB/s theo cách thể hiện qua average latency, p50 và xu hướng p99 tăng nhanh hơn throughput.

CHƯƠNG VII. Kịch bản thực nghiệm 2: Burst traffic và suy giảm mạng

7.1. Mục tiêu

Kịch bản thực nghiệm thứ hai được xây dựng nhằm khảo sát ảnh hưởng của suy giảm mạng đến hiệu năng của cụm Redpanda trong điều kiện burst traffic. Nếu kịch bản 1 tập trung vào giới hạn chịu tải trong điều kiện mạng bình thường, thì kịch bản 2 được dùng để quan sát hệ thống khi lưu lượng đến không liên tục và khi đường truyền giữa các broker bị làm xấu một cách có chủ đích.

Mục tiêu chính của kịch bản này là đánh giá mức độ thay đổi của throughput, average latency và tail latency khi so sánh giữa hai điều kiện:

- không áp dụng network impairment;
- có áp dụng network impairment bằng tc netem.

Bên cạnh đó, kịch bản này còn nhằm so sánh tác động của hai cấu hình xác nhận dữ liệu acks=1 và acks=all. Việc so sánh này có ý nghĩa quan trọng vì trong hệ thống phân tán có replication, cấu hình xác nhận dữ liệu quyết định trực tiếp thời điểm producer nhận được phản hồi thành công, từ đó ảnh hưởng đến độ trễ và khả năng duy trì throughput khi mạng bị suy giảm.

Từ góc nhìn đánh giá hiệu năng hệ thống mạng, kịch bản 2 được dùng để trả lời hai câu hỏi: thứ nhất, suy giảm mạng làm hiệu năng của Redpanda thay đổi như thế nào trong điều kiện burst traffic; thứ hai, mức độ nhạy cảm của acks=all so với acks=1 trước biến động của mạng nội bộ giữa các broker là bao nhiêu.

7.2. Thiết lập

Kịch bản 2 được thực hiện trên cùng cụm Redpanda gồm ba broker như ở Chương IV. Nhóm xây dựng burst traffic theo chu kỳ 10 giây ON / 20 giây OFF, lặp lại trong nhiều burst liên tiếp

để tạo ra các pha tải tăng đột ngột rồi giảm xuống. Cách tổ chức này giúp mô phỏng tốt hơn hành vi của các hệ thống thực tế, nơi lưu lượng đến thường không hoàn toàn đều theo thời gian.

Nhóm thực hiện bốn nhóm chạy chính:

- Không netem + acks=1
- Không netem + acks=all
- Có netem + acks=1
- Có netem + acks=all

Trong các lần chạy có suy giảm mạng, nhóm sử dụng tc netem trên các broker để mô phỏng việc tăng độ trễ, giới hạn băng thông và thêm mất gói ở mức nhẹ trên vùng lưu lượng replication giữa các broker. Mục tiêu của cấu hình này là tạo ra tác động rõ rệt đến hiệu năng nhưng vẫn giữ cho cụm hoạt động đủ ổn định để đo kiểm trong môi trường ảo hóa VMware.

Cấu hình tc netem được áp dụng trên interface ens33 của mỗi broker với các tham số cụ thể như sau:

```
sudo tc qdisc del dev ens33 root
sudo tc qdisc add dev ens33 root handle 1: tbf rate 50mbit
burst 32kbit latency 400ms
sudo tc qdisc add dev ens33 parent 1:1 handle 10: netem
delay 30ms loss 0.05%
```

Trong đó, băng thông được giới hạn ở mức 50 Mbit/s, độ trễ bổ sung là 30ms và tỉ lệ mất gói là 0.05%. Các giá trị này được chọn để tạo ra suy giảm đủ rõ rệt nhưng vẫn giữ cho cụm duy trì kết nối ổn định trong suốt quá trình đo kiểm.

Mỗi điều kiện được thực hiện qua nhiều cycle, sau đó nhóm lấy giá trị trung bình để giảm ảnh hưởng của dao động ngẫu nhiên giữa các lần chạy. Các chỉ số được sử dụng trong phân tích gồm throughput trung bình, average latency, p99 và p99.9. Ngoài ra, nhóm cũng thu thập consumer log và lag log để hỗ trợ quan sát định tính, tuy nhiên các kết luận định lượng chính trong chương này vẫn dựa trên throughput và latency.

7.3. Kỳ vọng

Về mặt lý thuyết, khi burst traffic được áp vào hệ thống, throughput ở pha ON sẽ chịu tác động trực tiếp của tốc độ xử lý của broker và cơ chế replication giữa các replica. Nếu mạng nội bộ giữa các broker vẫn ổn định, hệ thống có thể hấp thụ burst tốt hơn và duy trì được độ trễ thấp hơn. Ngược lại, khi mạng bị suy giảm, thời gian replication tăng lên, khiến độ trễ phản hồi cho producer tăng và throughput hiệu dụng giảm.

Trong cấu hình acks=1, producer chỉ cần chờ leader xác nhận dữ liệu, nên kỳ vọng độ nhạy với impairment của mạng sẽ thấp hơn. Trong cấu hình acks=all, producer chỉ nhận ACK sau khi leader thu đủ xác nhận từ các replica cần thiết. Do đó, kỳ vọng của nhóm là acks=all sẽ chịu ảnh hưởng mạnh hơn khi mạng nội bộ giữa các broker bị tăng độ trễ hoặc mất gói.

Từ đó, nhóm kỳ vọng ba xu hướng chính:

- Throughput sẽ giảm khi áp dụng network impairment.
- Average latency, p99 và p99.9 sẽ tăng rõ rệt khi mạng bị suy giảm.

- Cấu hình acks=all sẽ thể hiện mức suy giảm lớn hơn hoặc nhạy cảm hơn so với acks=1 khi đặt trong cùng điều kiện mạng xấu.

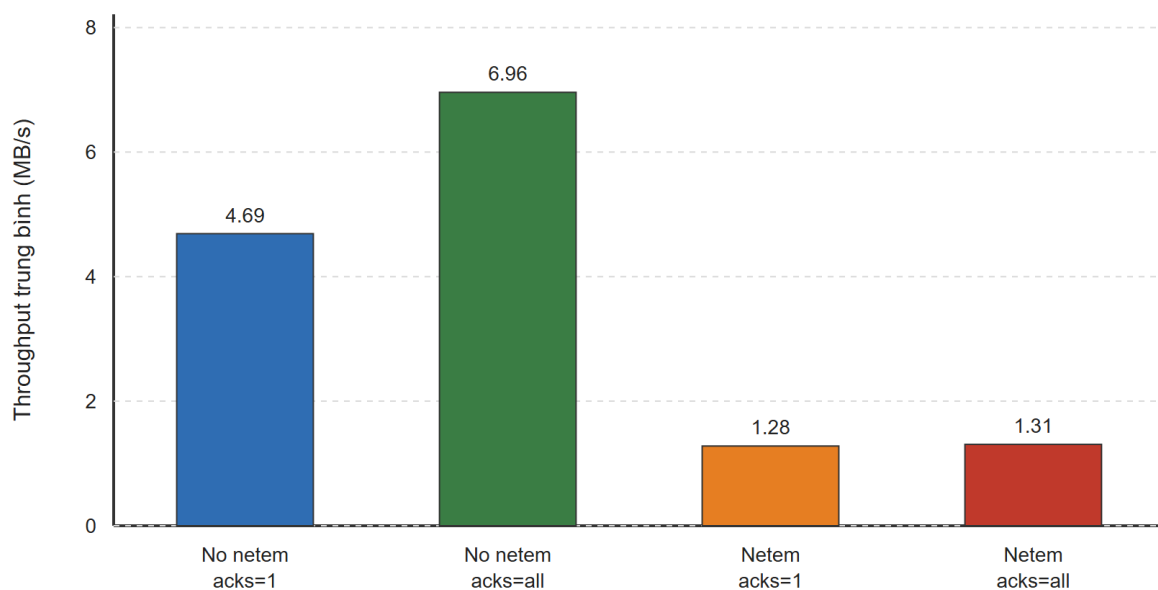
7.4. Kết quả và phân tích

Bảng 5. Trình bày giá trị trung bình của bốn điều kiện thực nghiệm trong kịch bản 2.

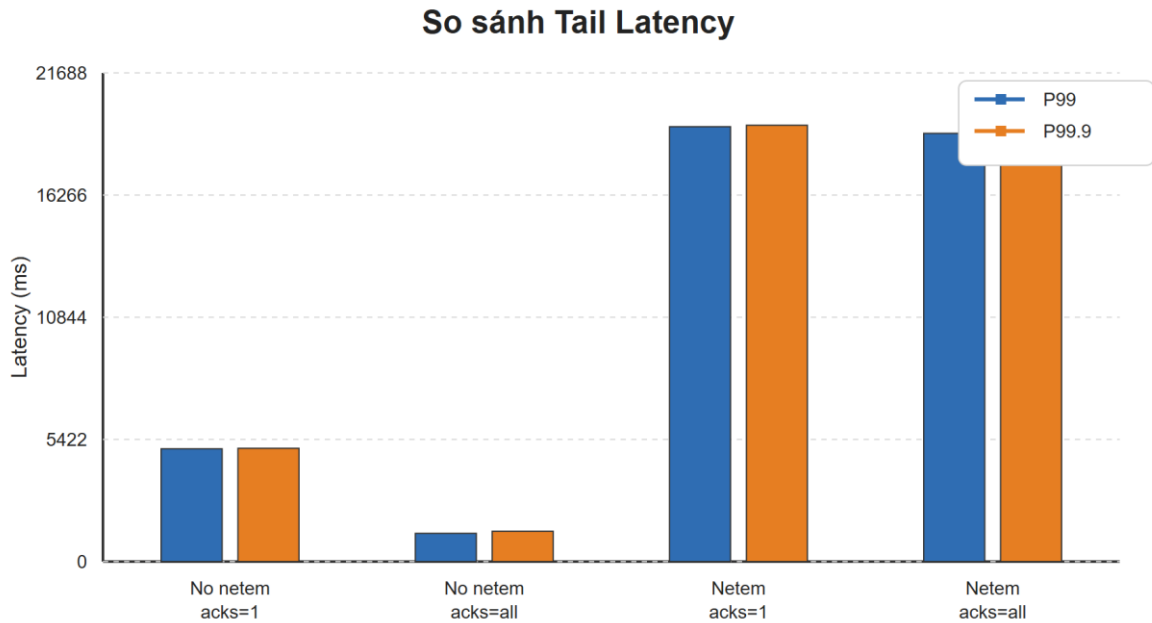
Điều kiện	Throughput (MB/s)	Avg latency (ms)	p99 (ms)	p99.9 (ms)
Không netem + acks=1	4.69	3420.18	5005.33	5027.67
Không netem + acks=all	6.96	660.61	1252.33	1345.00
Có netem + acks=1	1.28	15288.26	19301.33	19364.67
Có netem + acks=all	1.31	15118.51	19005.00	19061.33

Các số liệu tổng quát cho thấy việc áp dụng suy giảm mạng làm hiệu năng của hệ thống giảm mạnh ở cả hai cấu hình xác nhận dữ liệu. Điều này thể hiện rõ trên cả throughput lẫn các chỉ số latency ở phần đuôi phân bố.

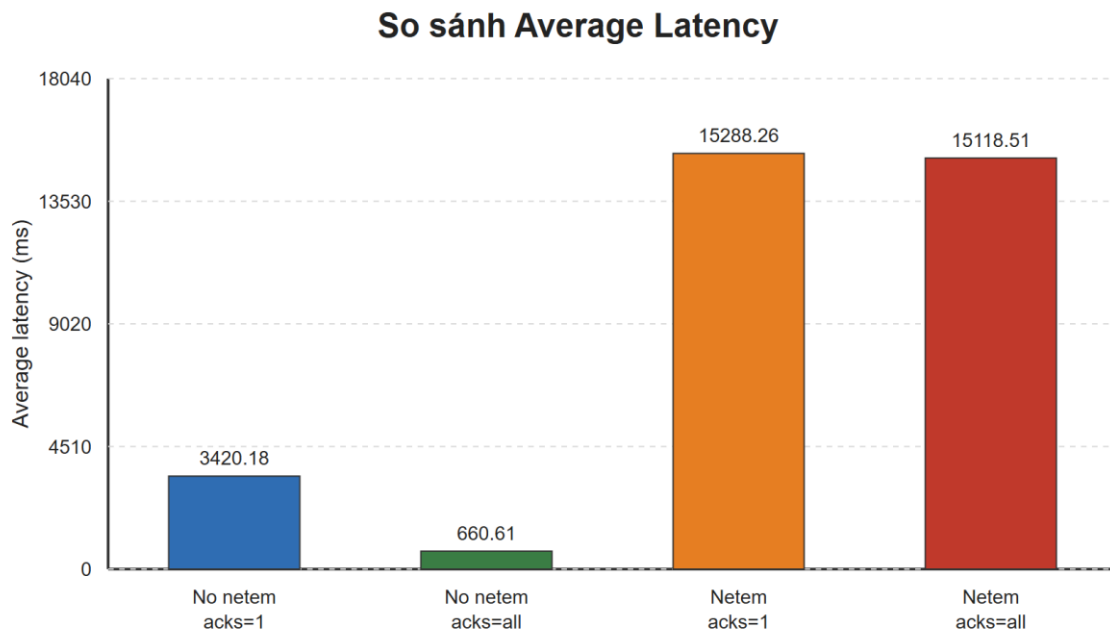
So sánh Throughput Trung bình



Hình 12. So sánh throughput trung bình giữa các điều kiện thực nghiệm.



Hình 13. So sánh tail latency giữa các điều kiện thực nghiệm.



Hình 14. So sánh average latency giữa các điều kiện thực nghiệm.

7.4.1. Ảnh hưởng của suy giảm mạng đến throughput

Hình 6.1 cho thấy throughput giảm mạnh khi áp dụng network impairment. Đối với cấu hình acks=1, throughput trung bình giảm từ 4.69 MB/s xuống 1.28 MB/s, tương đương mức giảm khoảng 72.7%. Đối với cấu hình acks=all, throughput giảm từ 6.96 MB/s xuống 1.31 MB/s, tương đương khoảng 81.2%.

Như vậy, dù cả hai cấu hình đều chịu ảnh hưởng lớn khi mạng bị suy giảm, acks=all có mức suy giảm tương đối mạnh hơn so với chính baseline của nó. Điều này phù hợp với cơ chế hoạt động của Redpanda: trong cấu hình acks=all, producer không chỉ phụ thuộc vào leader mà còn phụ thuộc vào quá trình replication và xác nhận từ các replica khác. Khi độ trễ mạng tăng hoặc băng thông replication bị bóp lại, thời gian hoàn tất một lần ghi tăng lên, dẫn đến throughput hiệu dụng giảm rõ rệt.

7.4.2. Ảnh hưởng của suy giảm mạng đến tail latency

Hình 6.2 cho thấy tác động của suy giảm mạng lên tail latency là rất lớn. Trong điều kiện không netem, acks=1 có p99 = 5005.33 ms, còn acks=all có p99 = 1252.33 ms. Khi bật netem, p99 tăng lên lần lượt 19301.33 ms và 19005.00 ms.

Nếu so với baseline tương ứng, acks=1 có p99 tăng khoảng 3.86 lần, trong khi acks=all tăng khoảng 15.2 lần. Với p99.9, xu hướng cũng tương tự: acks=1 tăng từ 5027.67 ms lên 19364.67 ms, còn acks=all tăng từ 1345.00 ms lên 19061.33 ms.

Kết quả này cho thấy tail latency là chỉ số rất nhạy với suy giảm mạng. Trong điều kiện burst traffic, khi hệ thống phải xử lý các pha tải tăng đột ngột, bất kỳ độ trễ bổ sung nào trên đường replication giữa các broker đều có thể làm những yêu cầu chậm nhất tăng lên rất mạnh. Điều này đặc biệt rõ với acks=all, vì mỗi lần ghi cần chờ thêm bước xác nhận từ quorum.

7.4.3. Ảnh hưởng của suy giảm mạng đến average latency

Hình 6.3 cho thấy average latency cũng tăng mạnh khi mạng bị suy giảm. Với acks=1, average latency tăng từ 3420.18 ms lên 15288.26 ms, tức khoảng 4.47 lần. Với acks=all, average latency tăng từ 660.61 ms lên 15118.51 ms, tức khoảng 22.9 lần.

Mặc dù average latency không phản ánh được đầy đủ mức độ nghiêm trọng như p99 và p99.9, nó vẫn cho thấy hệ thống đã bị đẩy sang trạng thái xử lý chậm hơn rõ rệt khi có network impairment. Việc cả average latency và tail latency cùng tăng mạnh chứng tỏ tác động của impairment không chỉ xảy ra ở một vài yêu cầu bất thường, mà đã ảnh hưởng đến mặt bằng hiệu năng chung của toàn bộ hệ thống.

7.4.4. So sánh giữa acks=1 và acks=all

Một điểm đáng chú ý là trong điều kiện không netem, kết quả của acks=all không phải lúc nào cũng xấu hơn tuyệt đối so với acks=1. Cụ thể, trong bộ số liệu hiện tại, acks=all cho throughput cao hơn và latency thấp hơn so với baseline của acks=1. Kết quả này có phần ngược với kỳ vọng lý thuyết đơn giản, nhưng không nhất thiết đồng nghĩa với việc acks=all luôn hiệu quả hơn trong điều kiện bình thường. Nguyên nhân là thực nghiệm burst traffic vẫn chịu ảnh hưởng bởi dao động giữa các lần chạy, trạng thái cụm tại thời điểm đo và mức độ ổn định của từng baseline; trên thực tế, log của nhóm acks=1 baseline cũng cho thấy một số dấu hiệu bất ổn trong quá trình chạy. Vì vậy, báo cáo không diễn giải khác biệt baseline này theo hướng kết luận tuyệt đối về ưu thế của từng cấu hình ACK.

Thay vào đó, nếu so sánh theo hướng hợp lý hơn là so với chính baseline của từng cấu hình, có thể thấy acks=all nhạy với network impairment hơn đáng kể. Throughput của acks=all giảm nhiều hơn, average latency tăng nhiều hơn và p99 cũng tăng mạnh hơn. Vì vậy, kết luận phù hợp từ kịch bản 2 không phải là acks=all luôn chậm hơn trong mọi trường hợp, mà là acks=all dễ bị tổn thương hơn khi chất lượng mạng giữa các broker bị suy giảm.

7.4.5. Nhận xét tổng hợp cho kịch bản 2

Từ các kết quả trên có thể rút ra ba nhận xét chính.

Thứ nhất, network impairment có ảnh hưởng rất lớn đến hiệu năng của Redpanda trong điều kiện burst traffic. Khi mạng giữa các broker bị làm xấu, throughput giảm mạnh trong khi average latency, p99 và p99.9 cùng tăng lên ở mức rất lớn.

Thứ hai, tail latency là chỉ số phản ánh rõ nhất tác động của suy giảm mạng. So với throughput, p99 và p99.9 cho thấy rõ hơn việc hệ thống bị đẩy vào trạng thái phản hồi chậm dưới các pha tải burst khi đường replication không còn thuận lợi.

Thứ ba, cấu hình acks=all thể hiện độ nhạy cao hơn với network impairment nếu so sánh theo tương quan với chính baseline của nó. Điều này phù hợp với lý thuyết đã trình bày ở Chương II: khi producer phải chờ xác nhận từ quorum, mạng nội bộ giữa các broker trở thành một phần trực tiếp của đường găng xử lý.

Nhìn chung, kịch bản 2 cho thấy trong các hệ thống streaming có replication như Redpanda, chất lượng mạng nội bộ giữa các broker là yếu tố có ảnh hưởng quyết định đến cả throughput và tail latency, đặc biệt khi hệ thống vận hành dưới burst traffic và khi sử dụng cơ chế xác nhận dữ liệu chặt chẽ hơn.

CHƯƠNG VIII. Thảo luận tổng hợp

8.1. Tác động của tải đến throughput và latency

Kết quả của kịch bản 1 cho thấy throughput và latency không biến thiên độc lập, mà có quan hệ đánh đổi rõ ràng khi hệ thống tiến gần vùng bão hòa. Ở vùng tải thấp và trung bình, Redpanda vẫn duy trì được tốc độ xử lý tốt, throughput tăng tương ứng với offered load và các chỉ số latency ở mức chấp nhận được. Tuy nhiên, khi tải tiếp tục tăng và tiến gần giới hạn xử lý hiệu dụng, throughput bắt đầu tăng chậm lại hoặc suy giảm, trong khi latency, đặc biệt là p99, tăng mạnh theo hướng phi tuyến.

Hiện tượng này phù hợp với lý thuyết hàng đợi đã trình bày ở Chương II. Khi tốc độ đến λ tiến gần tốc độ phục vụ μ , hệ số sử dụng ρ tiến gần 1, làm thời gian chờ trong hệ thống tăng nhanh. Trong bối cảnh streaming, sự gia tăng này được phản ánh qua việc hàng đợi nội bộ và độ trễ đuôi cùng tăng lên, dù throughput chưa sụt giảm ngay lập tức ở mọi trường hợp. Vì vậy, latency, đặc biệt là tail latency, là chỉ báo nhạy hơn throughput trong việc nhận biết hệ thống đang tiến vào vùng mất ổn định.

Kết quả cũng cho thấy kích thước bản tin ảnh hưởng rõ rệt đến cách hệ thống tiến tới saturation. Với nhóm 256B, throughput đạt mức cao và duy trì trong một vùng tải tương đối rộng, nhưng latency vẫn tăng đáng kể khi tải cao. Với nhóm 1KB, knee point biểu hiện rõ hơn qua việc throughput giảm mạnh và p99 tăng đột biến ở vùng 10–12 MB/s. Điều này cho thấy message size không chỉ làm thay đổi throughput tuyệt đối, mà còn làm thay đổi cách hệ thống phản ứng khi chịu tải lớn.

8.2. Tác động của mạng đến cơ chế replication

Kịch bản 2 cho thấy chất lượng mạng giữa các broker có ảnh hưởng trực tiếp đến hiệu năng của Redpanda trong điều kiện burst traffic. Khi mạng nội bộ bị làm xấu bằng tc netem, throughput giảm mạnh, đồng thời average latency, p99 và p99.9 đều tăng đáng kể. Điều này phản ánh vai trò quan trọng của đường replication trong một hệ thống streaming có nhiều replica.

Về mặt cơ chế, mỗi lần ghi dữ liệu trong Redpanda không chỉ là quá trình producer gửi dữ liệu đến leader, mà còn liên quan đến việc leader phân phối dữ liệu sang các follower và chờ xác nhận tùy theo cấu hình acks. Khi mạng giữa các broker bị tăng độ trễ, bị giới hạn băng thông hoặc có mất gói, thời gian hoàn tất replication tăng lên. Kết quả là độ trễ phản hồi cho producer cũng tăng, đồng thời khả năng duy trì throughput ổn định giảm xuống.

Trong bối cảnh burst traffic, tác động này còn rõ hơn vì hệ thống phải xử lý các pha tải tăng đột ngột. Khi tốc độ dữ liệu đến tăng nhanh trong pha ON, bất kỳ độ trễ bổ sung nào trên đường replication cũng dễ làm hàng đợi tích tụ nhanh hơn. Do đó, mạng không chỉ là thành phần hỗ trợ truyền dữ liệu, mà trở thành một phần trực tiếp của đường găng hiệu năng trong hệ thống phân tán có replication.

8.3. Trade-off giữa hiệu năng và độ tin cậy

Một kết quả quan trọng của báo cáo là làm rõ trade-off giữa hiệu năng và độ tin cậy dữ liệu thông qua hai cấu hình acks=1 và acks=all. Về nguyên lý, acks=1 cho phép producer nhận phản hồi sớm hơn vì chỉ cần leader xác nhận dữ liệu, trong khi acks=all yêu cầu leader thu đủ xác nhận từ quorum. Do đó, acks=all cung cấp mức đảm bảo dữ liệu cao hơn nhưng đồng thời cũng làm hệ thống phụ thuộc nhiều hơn vào tốc độ replication và chất lượng mạng giữa các broker.

Kết quả thực nghiệm cho thấy điều này không phải lúc nào cũng thể hiện bằng việc acks=all luôn có latency cao hơn tuyệt đối trong mọi lần chạy. Tuy nhiên, nếu so sánh theo tương quan với chính baseline của từng cấu hình, có thể thấy acks=all nhạy hơn rõ rệt với network impairment. Khi mạng bị suy giảm, throughput của acks=all giảm nhiều hơn và các chỉ số latency cũng tăng mạnh hơn so với acks=1.

Từ góc nhìn vận hành hệ thống, điều này cho thấy việc lựa chọn cấu hình xác nhận dữ liệu không thể tách rời điều kiện hạ tầng mạng. Nếu ưu tiên độ tin cậy cao và sử dụng acks=all, hệ thống cần một môi trường mạng nội bộ ổn định hơn để tránh tác động quá lớn lên throughput và tail latency. Ngược lại, nếu ưu tiên hiệu năng trong môi trường tài nguyên hạn chế, acks=1 có thể mang lại lợi thế về độ trễ, nhưng phải chấp nhận mức đảm bảo dữ liệu thấp hơn.

CHƯƠNG IX. Kết luận

9.1. Kết luận chính

Báo cáo đã thực hiện đánh giá hiệu năng nền tảng message streaming Redpanda trên môi trường gồm ba broker ảo hóa, thông qua hai kịch bản chính là tải tăng dần và burst traffic kết hợp suy giảm mạng. Các kết quả thực nghiệm cho thấy Redpanda có thể duy trì throughput tốt trong vùng tải ổn định, nhưng khi offered load tiến gần năng lực xử lý hiệu dụng, hệ thống

bắt đầu xuất hiện knee point, nơi latency tăng mạnh và throughput không còn tăng tương ứng với tải vào.

Đối với kịch bản tải tăng dần, nhóm 1KB cho thấy hiện tượng bão hòa rõ rệt hơn so với nhóm 256B. Vùng 10–11 MB/s được xác định là khoảng giá trị mà hệ thống bắt đầu mất ổn định, thể hiện qua throughput suy giảm, p99 tăng đột biến và sự xuất hiện của các run lỗi. Điều này cho thấy tail latency là chỉ số đặc biệt quan trọng trong việc nhận diện vùng vận hành không an toàn của hệ thống.

Đối với kịch bản burst traffic kết hợp suy giảm mạng, kết quả cho thấy network impairment có ảnh hưởng rất lớn đến cả throughput và latency của Redpanda. Khi chất lượng mạng giữa các broker bị làm xấu, throughput giảm mạnh trong khi average latency, p99 và p99.9 tăng đáng kể. Ngoài ra, cấu hình acks=all thể hiện độ nhạy cao hơn với suy giảm mạng nếu so sánh theo tương quan với baseline của chính nó, cho thấy mạng nội bộ giữa các broker là yếu tố rất quan trọng đối với hiệu năng của cơ chế replication.

Tổng hợp lại, báo cáo cho thấy hiệu năng của Redpanda không chỉ phụ thuộc vào offered load và kích thước bản tin, mà còn phụ thuộc mạnh vào cơ chế xác nhận dữ liệu và chất lượng mạng giữa các broker. Đây là những yếu tố cần được cân nhắc đồng thời khi triển khai và vận hành các hệ thống streaming phân tán trong thực tế.

9.2. Hạn chế

Một hạn chế quan trọng của nghiên cứu là toàn bộ thực nghiệm được triển khai trên môi trường ảo hóa VMware thay vì hạ tầng vật lý. Do đó, kết quả có thể chịu ảnh hưởng của overhead từ hypervisor, tài nguyên dùng chung và đặc tính của mạng ảo, nên chưa phản ánh hoàn toàn hành vi của hệ thống trong môi trường production.

Bên cạnh đó, báo cáo chủ yếu sử dụng các chỉ số ở tầng ứng dụng như throughput và latency, trong khi một số chỉ số mức hệ thống như CPU utilization, disk I/O, mức sử dụng mạng hay retransmission chưa được thu thập đầy đủ. Vì vậy, phần phân tích bottleneck trong báo cáo chủ yếu dựa trên hành vi quan sát được từ các chỉ số đầu ra thay vì các số đo sâu ở tầng hệ điều hành và mạng.

Ngoài ra, một số phép đo liên quan đến consumer lag và recovery trong kịch bản burst traffic mới dừng ở mức hỗ trợ quan sát định tính, chưa được khai thác đầy đủ thành kết quả định lượng chính thức. Đây là giới hạn cần được lưu ý khi diễn giải kết quả.

9.3. Hướng phát triển

Trong các nghiên cứu tiếp theo, nhóm có thể mở rộng thực nghiệm theo ba hướng chính. Thứ nhất, triển khai hệ thống trên môi trường vật lý hoặc môi trường ảo hóa được kiểm soát tốt hơn để giảm ảnh hưởng của overhead từ hypervisor, từ đó nâng cao độ tin cậy của kết quả đo.

Thứ hai, bổ sung các chỉ số mức hệ thống như CPU utilization, disk I/O, network utilization và retransmission để phân tích bottleneck sâu hơn. Việc kết hợp các số liệu này với throughput và latency sẽ giúp giải thích rõ hơn nguyên nhân của knee point và cơ chế suy giảm hiệu năng khi mạng bị làm xấu.

Thứ ba, mở rộng tập kịch bản thực nghiệm bằng cách thay đổi thêm số lượng partition, replication factor, số producer/consumer đồng thời, hoặc mức độ suy giảm mạng khác nhau.

Điều này sẽ giúp đánh giá toàn diện hơn hành vi của Redpanda trong nhiều điều kiện vận hành và cung cấp cơ sở tốt hơn cho việc tối ưu cấu hình trong thực tế.

CHƯƠNG X. So sánh Redpanda với Apache Kafka và RabbitMQ

Chương này so sánh Redpanda với hai nền tảng messaging phổ biến là Apache Kafka và RabbitMQ, nhằm làm rõ vị trí của Redpanda trong hệ sinh thái và lý do lựa chọn nó làm đối tượng nghiên cứu.

10.1. Tổng quan

Apache Kafka (2011, LinkedIn) là nền tảng event streaming log-based, dữ liệu lưu theo retention và consumer đọc qua offset. Kafka dùng ZooKeeper hoặc KRaft để quản lý cluster, được viết bằng Java/Scala trên JVM.

RabbitMQ là message broker triển khai giao thức AMQP theo mô hình exchange-queue. Message bị xóa sau khi consumer ACK, không hỗ trợ replay. RabbitMQ phù hợp cho task distribution và routing phức tạp hơn là event streaming khối lượng lớn.

Redpanda (2020, Redpanda Data) tương thích hoàn toàn Kafka API nhưng được viết bằng C++ với Seastar framework và tích hợp Raft built-in, loại bỏ phụ thuộc vào JVM và ZooKeeper.

10.2. So sánh tổng hợp

Tiêu chí	Apache Kafka	RabbitMQ	Redpanda
Ngôn ngữ	Java/Scala (JVM)	Erlang	C++ (Seastar)
Mô hình	Log-based pub/sub	Exchange / Queue	Log-based pub/sub
Giao thức	Kafka Protocol	AMQP, MQTT, STOMP	Kafka Protocol
Quản lý cluster	ZooKeeper / KRaft	Mnesia built-in	Raft built-in
Lưu trữ message	Theo retention	Xóa sau ACK	Theo retention
Replay lịch sử	Có	Không	Có
Throughput	Rất cao	Trung bình	Rất cao
Tail latency tải cao	Spike do GC	Ổn định	Ổn định (không GC)
Độ phức tạp vận hành	Cao	Thấp	Thấp
Kafka API	Native	Không	Tương thích hoàn toàn

Bảng 6. So sánh tổng hợp giữa Apache Kafka, RabbitMQ và Redpanda.

10.3. Lý do lựa chọn Redpanda

Dựa trên bảng so sánh trên, nhóm chọn Redpanda vì ba lý do chính. Thứ nhất, Redpanda tương thích hoàn toàn Kafka API nên có thể dùng trực tiếp bộ công cụ Kafka CLI (kafka-producer-perf-test, kafka-consumer-groups) mà không cần thay đổi. Thứ hai, kiến trúc Raft built-in giúp triển khai cluster 3 node đơn giản hơn Kafka truyền thống, phù hợp với môi trường VMware

của nhóm. Thứ ba, Redpanda là nền tảng còn tương đối mới và chưa có nhiều nghiên cứu độc lập về hành vi dưới điều kiện mạng suy giảm — đây chính là khoảng trống mà báo cáo này hướng đến.

TÀI LIỆU THAM KHẢO

- [1] Microsoft Learn, “Event-driven architecture style.” [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/event-driven>. [Accessed: Apr. 2, 2026].
- [2] Redpanda, “How Redpanda Works.” [Online]. Available: <https://docs.redpanda.com/25.1/get-started/architecture/>. [Accessed: Apr. 2, 2026].
- [3] Redpanda, “Configure Producers.” [Online]. Available: <https://docs.redpanda.com/23.3/develop/produce-data/configure-producers/>. [Accessed: Apr. 2, 2026].
- [4] Redpanda, “Consumer Offsets.” [Online]. Available: <https://docs.redpanda.com/current/develop/consume-data/consumer-offsets/>. [Accessed: Apr. 2, 2026].
- [5] Redpanda, “Redpanda Self-Managed Quickstart.” [Online]. Available: <https://docs.redpanda.com/current/get-started/quick-start/>. [Accessed: Apr. 2, 2026].
- [6] Apache Kafka, “Downloads.” [Online]. Available: <https://kafka.apache.org/downloads>. [Accessed: Apr. 2, 2026].
- [7] Apache Kafka, “Quickstart.” [Online]. Available: <https://kafka.apache.org/quickstart>. [Accessed: Apr. 2, 2026].
- [8] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm (Extended Version),” 2014. [Online]. Available: <https://raft.github.io/raft.pdf>. [Accessed: Apr. 2, 2026].

